Experiment No-1

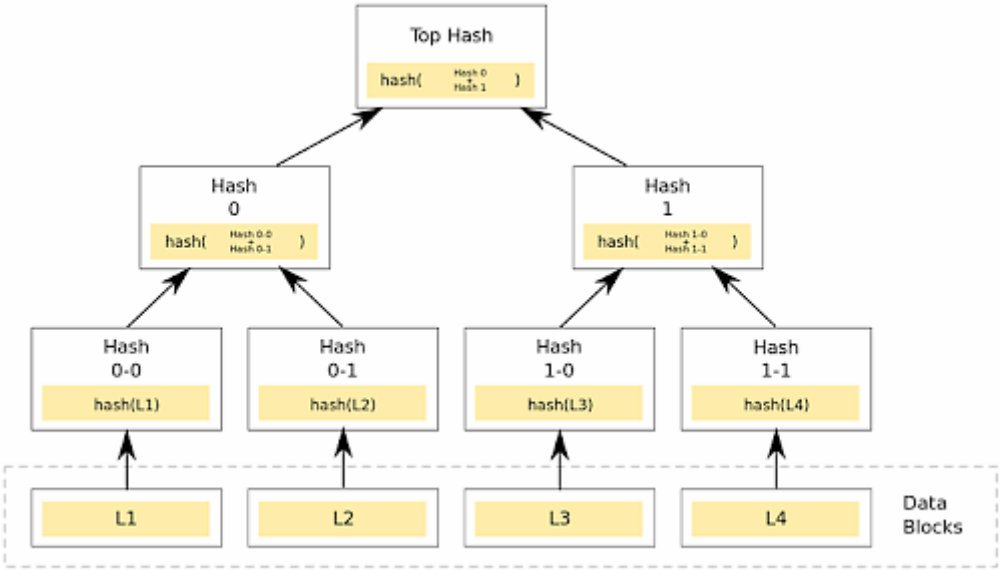
Cryptography in Blockchain, Merkle root tree hash

Cryptography is the process of developing a set of techniques and protocols to prevent any third party from gaining access to the data during a communication process. The word ‘cryptography’ is derived from two Greek terms, ‘Kryptos’ meaning ‘hidden’ and ‘Graphein’ meaning ‘to write’. Some terminologies used in cryptography: The method used to secure data from unauthorised access is called cryptography. Blockchain was built on the concept of enabling secure communication between two parties therefore, it uses cryptography to secure the transactions that take place between two nodes on the network. Blockchain uses two main concepts, cryptography and hashing. The former is used to encrypt messages in a peer-to-peer network, and the latter is used to secure information on the block and link the blocks in a blockchain.

Maintaining the security of participants and transactions as well as safeguarding against double-spending, is the primary focus of cryptography. It makes sure that the transactions on the blockchain network are secure by ensuring data can be obtained, read and processed only by individuals for whom it was intended.

What Is a Merkle Tree?

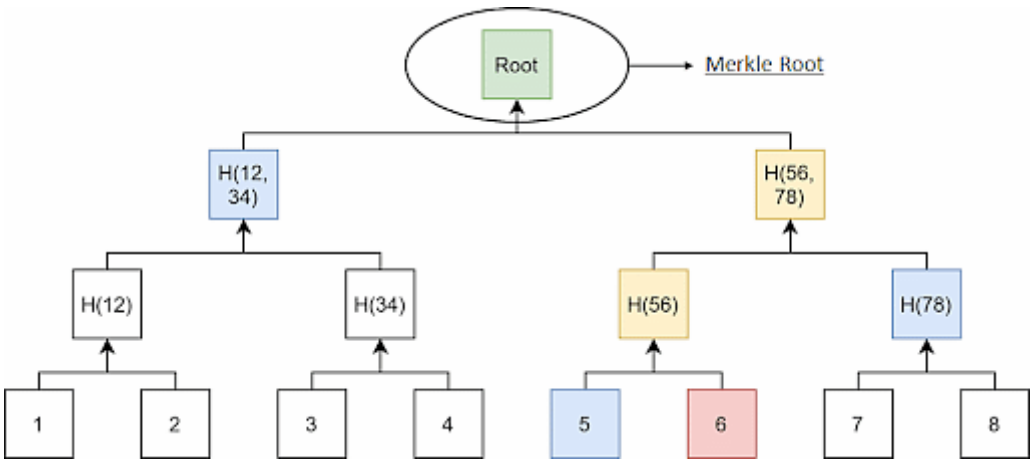
- Merkle trees, also known as Binary hash trees, are a prevalent sort of data structure in computer science.
- In bitcoin and other cryptocurrencies, they're used to encrypt blockchain data more efficiently and securely.
- It's a mathematical data structure made up of hashes of various data blocks that summarize all the transactions in a block.
- It also enables quick and secure content verification across big datasets and verifies the consistency and content of the data.



What Is a Merkle Root?

A Merkle root is a simple mathematical method for confirming the facts on a Merkle tree.They're used in cryptocurrency to ensure that data blocks sent through a peer-to-peer network are whole, undamaged, and unaltered.They play a very crucial role in the computation required to keep cryptocurrencies like bitcoin and ether running.

Merkle Tree in Blockchain:
A hash tree, also known as a Merkle tree, is a tree in which each leaf node is labeled with the cryptographic hash of a data block, and each non-leaf node is labeled with the cryptographic hash of its child nodes' labels. The majority of hash tree implementations are binary (each node has two child nodes), but they can also have many more child nodes.



Cryptographic Hash Functions

A hash function maps any type of arbitrary data of any length to a fixed-size output. It is commonly used in cryptography since it is a cryptographic function.

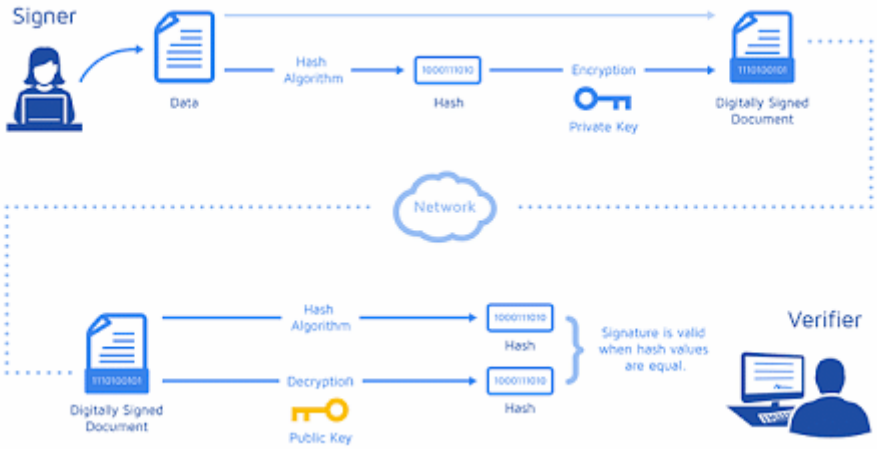
- They are efficient and are well-known for one property: they are irreversible. It's a one-way function that's only meant to work in one direction.
- Some of the Hash families available are Message Direct (MD), Secure Hash Function (SHF), and RIPE Message Direct (RIPEMD).

Now, take an example, if you use the SHA256 hash algorithm and pass 101Blockchains as input, you will get the following output

fbffd63a60374a31aa9811cbc80b577e23925a5874e86a17f712bab874f33ac9

Working of Merkle Trees

A Merkle tree totals all transactions in a block and generates a digital fingerprint of the entire set of operations, allowing the user to verify whether it includes a transaction in the block.



- Merkle trees are made by hashing pairs of nodes repeatedly until only one hash remains; this hash is known as the Merkle Root or the Root Hash.
- They're built from the bottom, using Transaction IDs, which are hashes of individual transactions.
- Each non-leaf node is a hash of its previous hash, and every leaf node is a hash of transactional data.

Benefits of Merkle Tree in Blockchain

Merkle trees provide four significant advantages -Validate the data's integrity: It can be used to validate the data's integrity effectively.

- Takes little disk space: Compared to other data structures, the Merkle tree takes up very little disk space.
- Tiny information across networks: Merkle trees can be broken down into small pieces of data for verification.
- Efficient Verification: The data format is efficient, and verifying the data's integrity takes only a few moments.

Why Is It Essential to Blockchain?

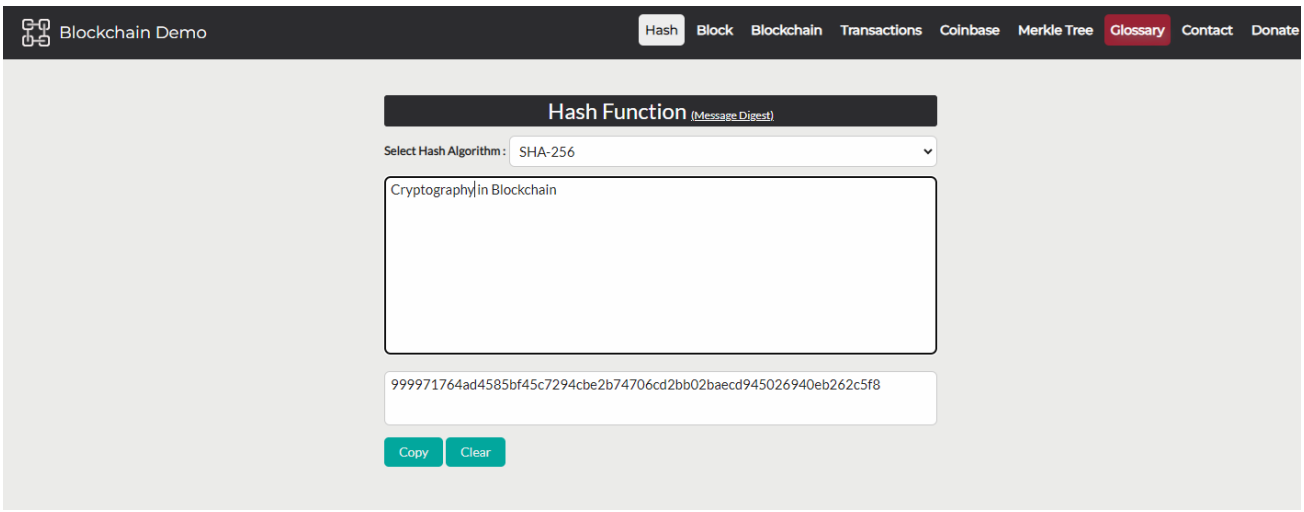
Think of a blockchain without Merkle Trees to get a sense of how vital they are for blockchain technology. Let’s have one of Bitcoin because its use of Merkle Trees is essential for the cryptocurrency and easier to grasp.

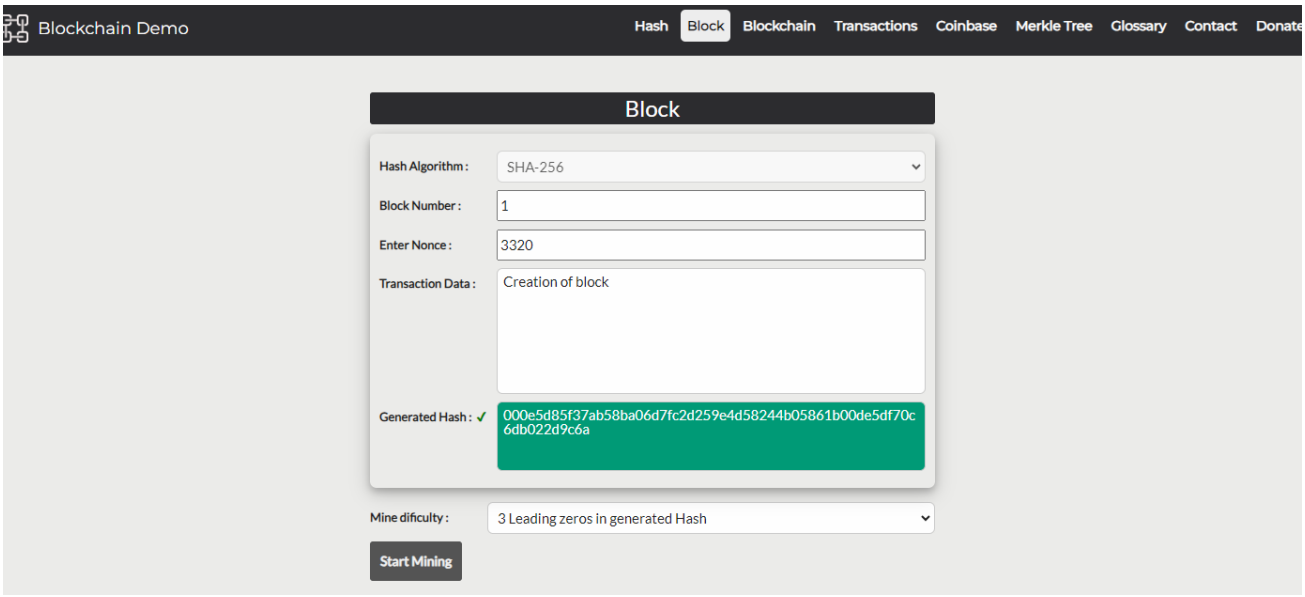
- If Bitcoin didn't include Merkle Trees, per se, every node on the network would have to retain a complete copy of every single Bitcoin transaction ever made. One can imagine how much information that would be.
- Any authentication request on Bitcoin would require an enormous amount of data to be transferred over the network: therefore, you'll need to validate the data on your own.
- To confirm that there were no modifications, a computer used for validation would need a lot of computing power to compare ledgers.

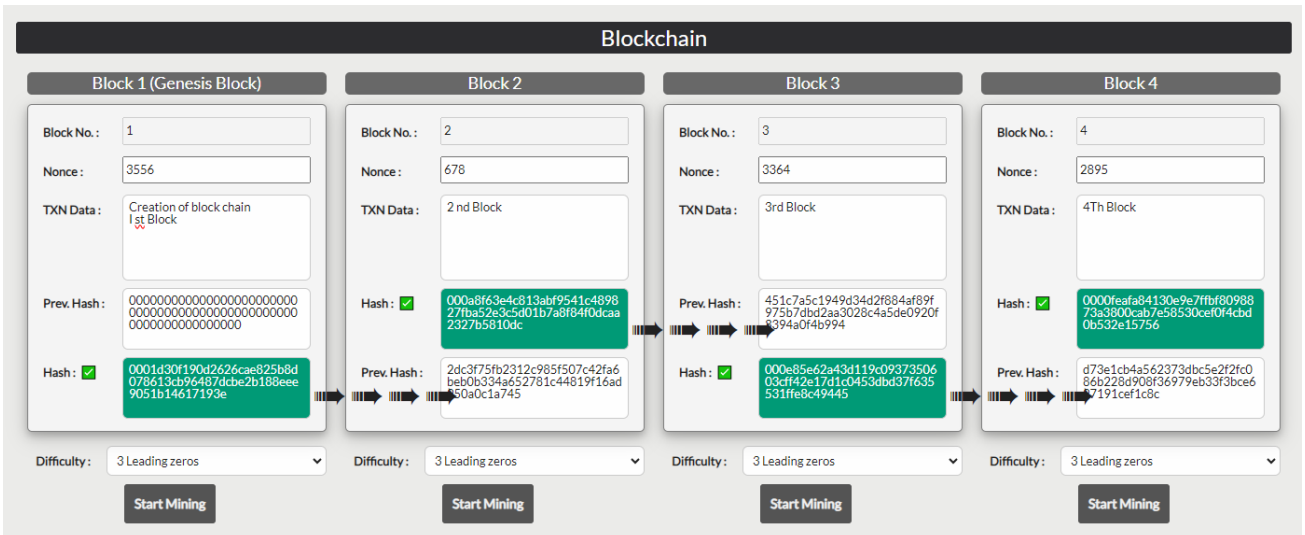
Merkle Trees are a solution to this issue. They hash records in accounting, thereby separating the proof of data from the data itself.

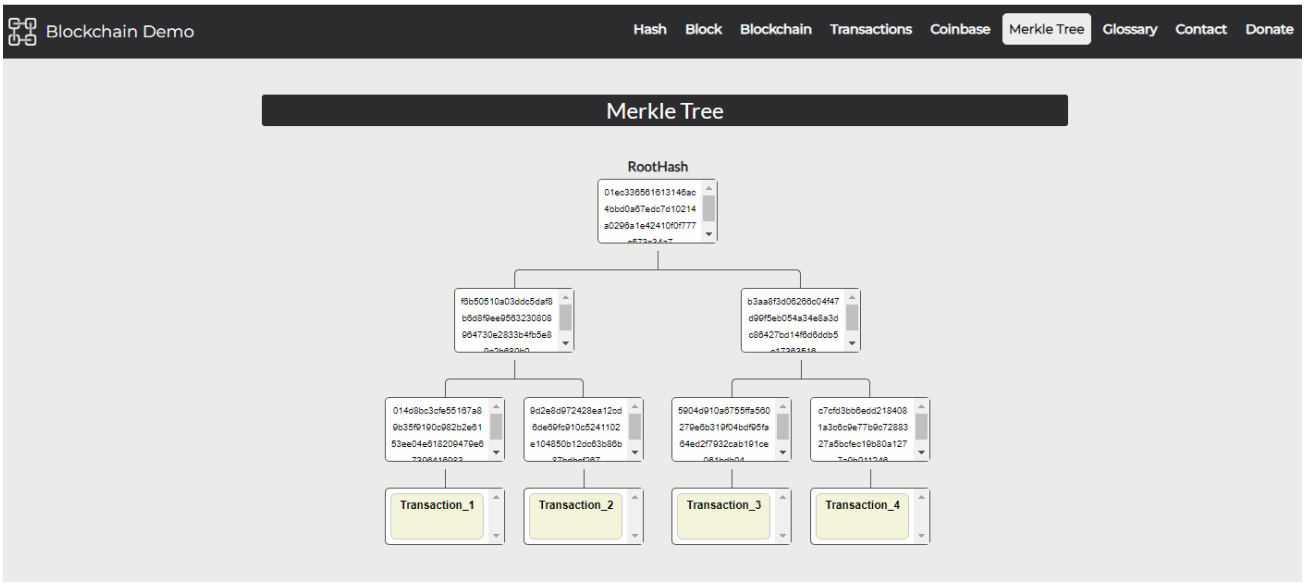
- Proving that giving tiny amounts of information across the network is all that is required for a transaction to be valid.
- Furthermore, it enables you to demonstrate that both ledger variations are identical in terms of nominal computer power and network bandwidth.

Output


Blockchain Demo
Hash
Block
Blockchain
Transactions
Coinbase
Merkle Tree
Glossary
Contact
Donate
Hash Function (Message Digest)
Select Hash Algorithm :
SHA-256
Cryptography in Blockchain
999971764ad4585bf45c7294cbe2b74706cd2bb02baecd945026940eb262c5f8
Copy
Clear


Blockchain Demo
Hash
Block
Blockchain
Transactions
Coinbase
Merkle Tree
Glossary
Contact
Donate
Block
Hash Algorithm :
SHA-256
Block Number :
1
Enter Nonce :
3320
Transaction Data :
Creation of block
Generated Hash :
000e5d85f37ab58ba06d7fc2d259e4d58244b05861b00de5df70c6db022d9c6a
Mine dificulty :
3 Leading zeros in generated Hash
Start Mining


Blockchain
Block 1 (Genesis Block)
Block No. :
1
Nonce :
3556
TXN Data :
Creation of block chain
I st Block
Prev. Hash :
0000000000000000000000000000000000000000000000000000000000000000
Hash :
0001d30f190d2626cae825b8d078613cb96487dcbe2b188eee9051b14617193e
Difficulty :
3 Leading zeros
Start Mining
Block 2
Block No. :
2
Nonce :
678
TXN Data :
2 nd Block
Hash :
000a8f63e4c813abf9541c489827fba52e3c5d01b7a8f84f0dcaa2327b5810dc
Prev. Hash :
2dc3f75fb2312c985f507c42fa6beb0b334a652781c44819f16ad650a0c1a745
Difficulty :
3 Leading zeros
Start Mining
Block 3
Block No. :
3
Nonce :
3364
TXN Data :
3rd Block
Prev. Hash :
451c7a5c1949d34d2f884af89f975b7dbd2aa3028c4a5de0920f8394a0f4b994
Hash :
000e85e62a43d119c0937350603cff42e17d1c0453dbd37f635531ffe8c49445
Difficulty :
3 Leading zeros
Start Mining
Block 4
Block No. :
4
Nonce :
2895
TXN Data :
4Th Block
Hash :
0000feafa84130e9e7ffbf8098873a3800cab7e58530cef0f4cbd0b532e15756
Prev. Hash :
d73e1cb4a562373dbc5e2f2fc086b228d908f36979eb33f3bce657191cef1c8c
Difficulty :
3 Leading zeros
Start Mining



Conclusion: Thus we have successfully studied cryptography in block using merkle tree.

Experiment No. 2.

Aim : Creating Smart Contract Using Solidity & Remix IDE.

Theory :

One of the most revolutionary features of blockchain is its ability to execute smart contracts — self-executing programs that run on blockchain without requiring intermediaries.

A smart contract contains rules written in code that are automatically enforced when predefined conditions are met. This eliminates the need for trust between parties & ensures transparency, immutability & automation.

Solidity Programming Language :

Solidity is a primary programming language for writing smart contracts on the Ethereum blockchain. It is statically typed, supports inheritance, libraries & complex user-defined types, & is heavily influenced by languages such as Java, C++, Python.

Key Features of Solidity :

- Version Pragma : Defines the compiler version (pragma solidity ^0.8.26) to ensure compatibility.
- License Identifier : Using SPDX identifiers like `// SPDX-License-Identifier: MIT` helps indicate the license type.
- Functions : Can be public, private, internal or external & state mutability like pure, view or payable.
- Data Types : supports Integers (int, uint), booleans, addresses, strings, & custom structs.
- Modifiers : Restrict Access or apply conditions before function execution.

Development Environment : Remix IDE.

Remix IDE is an open-source, web-based development environment for writing, compiling, deploying & testing solidity smart contracts.

Advantages of Remix IDE:

- Runs directly in browser, no installation required.
- Provides solidity compiler with version selection.
- Includes integrated debugger for tracing transactions.
- Offers simulated Ethereum environment for testing (JavaScript VM)

Basic Workflow in Remix IDE:

1. Write the contract: Create a .sol file & write solidity code.
2. Compile: Use the Solidity compiler in Remix IDE to detect syntax or logical errors.
3. Deploy: Choose the environment (JS VM, Injected Web3 or Web3 Provider) & deploy the contract.
4. Interact: Call functions & observe outputs directly in Remix.

State Mutability in Solidity :

Function in Solidity can be classified based on their interaction with the blockchain state :

- Pure : Does not read or modify the state
- View : Reads the state but does not modify.
- Payable : Allows function to receive Ether.

Gas Usage & Optimization :

Gas is the unit of computational cost in Ethereum. Writing optimized code ensures lower transaction fees.

- Avoid recursion for larger inputs
- Use appropriate data types when possible.
- Reduce unnecessary storage writes since storage is costly.

Security considerations :

While the given examples are computationally safe, real-world smart contracts must consider :

- Overflow / Underflow : Handles in Solidity ≥ 0.8.0 automatically.

- Reentrancy Attack : Use checks-effects-interactions pattern.
- Access Control : Apply onlyOwner modifiers or role-based permissions.
- Input Validation : Always check user inputs to prevent unexpected behavior.

Conclusion :

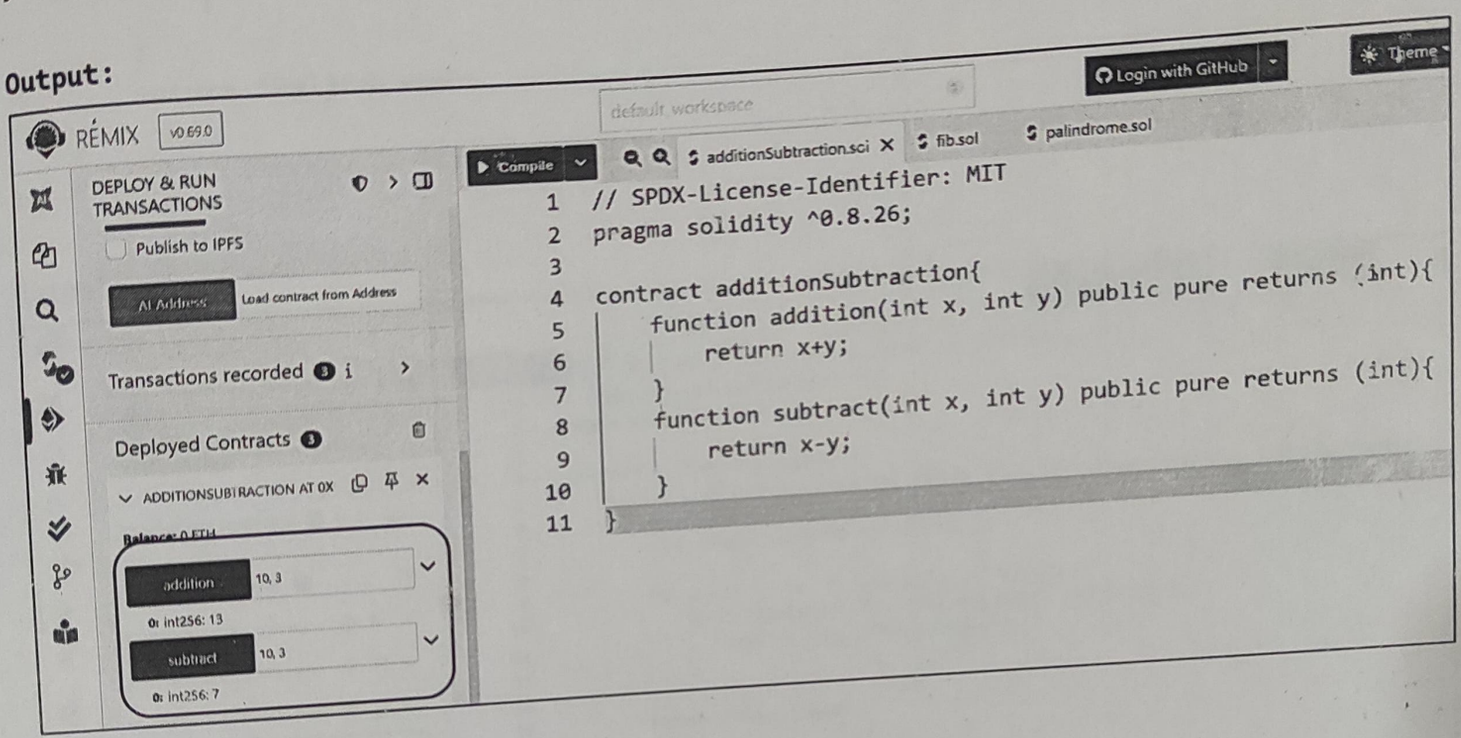
Creating smart contracts in Solidity using Remix IDE provides a practical & interactive way to understand blockchain programming. These contracts, while simple introduce the fundamental blocks for more complex decentralized applications (DApps).

Program Code:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract additionSubtraction{
    function addition(int x, int y) public pure returns (int){
        return x+y;
    }
    function subtract(int x, int y) public pure returns (int){
        return x-y;
    }
}
```

Output:



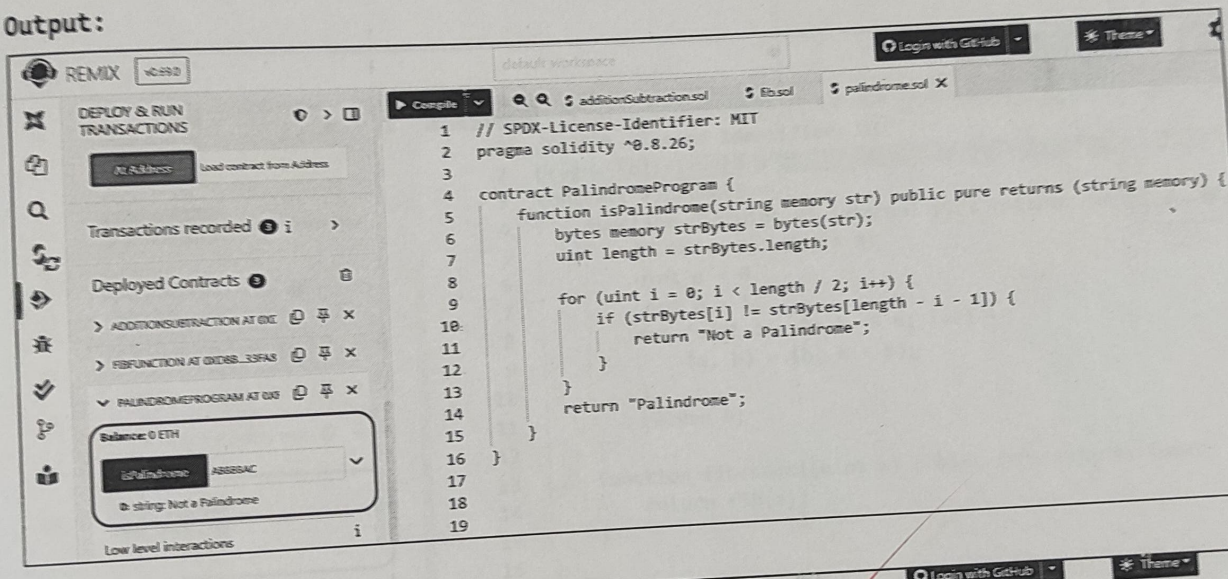
Program Code:

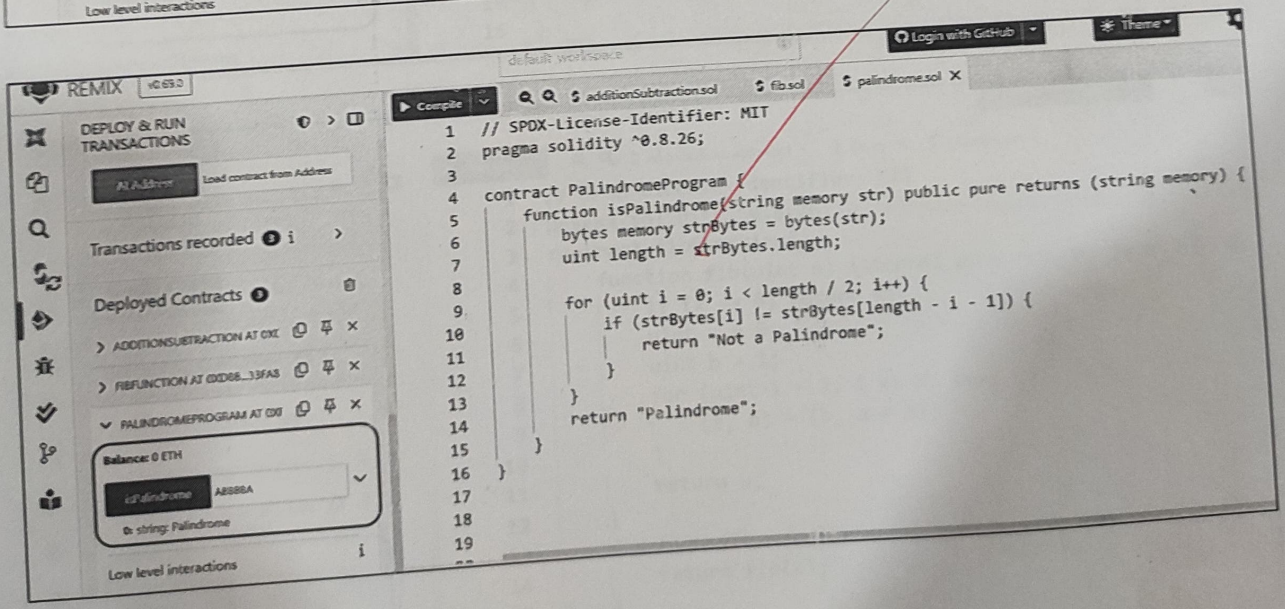
```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract PalindromeProgram {
    function isPalindrome(string memory str) public pure returns (string memory) {
        bytes memory strBytes = bytes(str);
        uint length = strBytes.length;

        for (uint i = 0; i < length / 2; i++) {
            if (strBytes[i] != strBytes[length - i - 1]) {
                return "Not a Palindrome";
            }
        }
        return "Palindrome";
    }
}
```

Output:



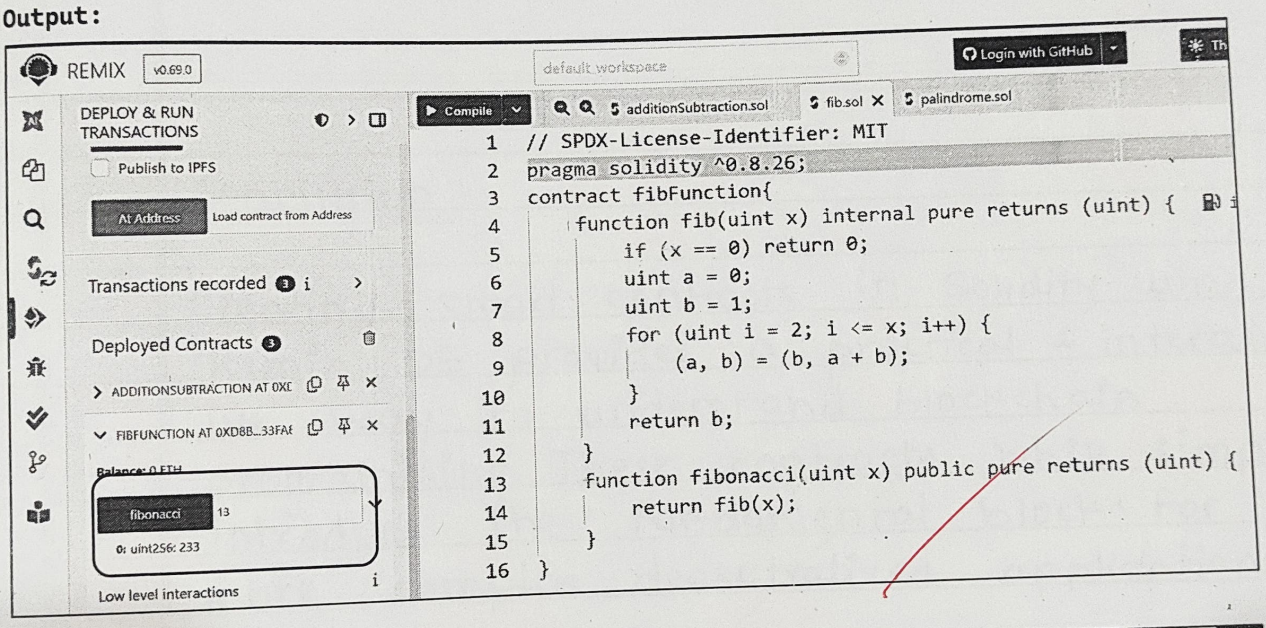


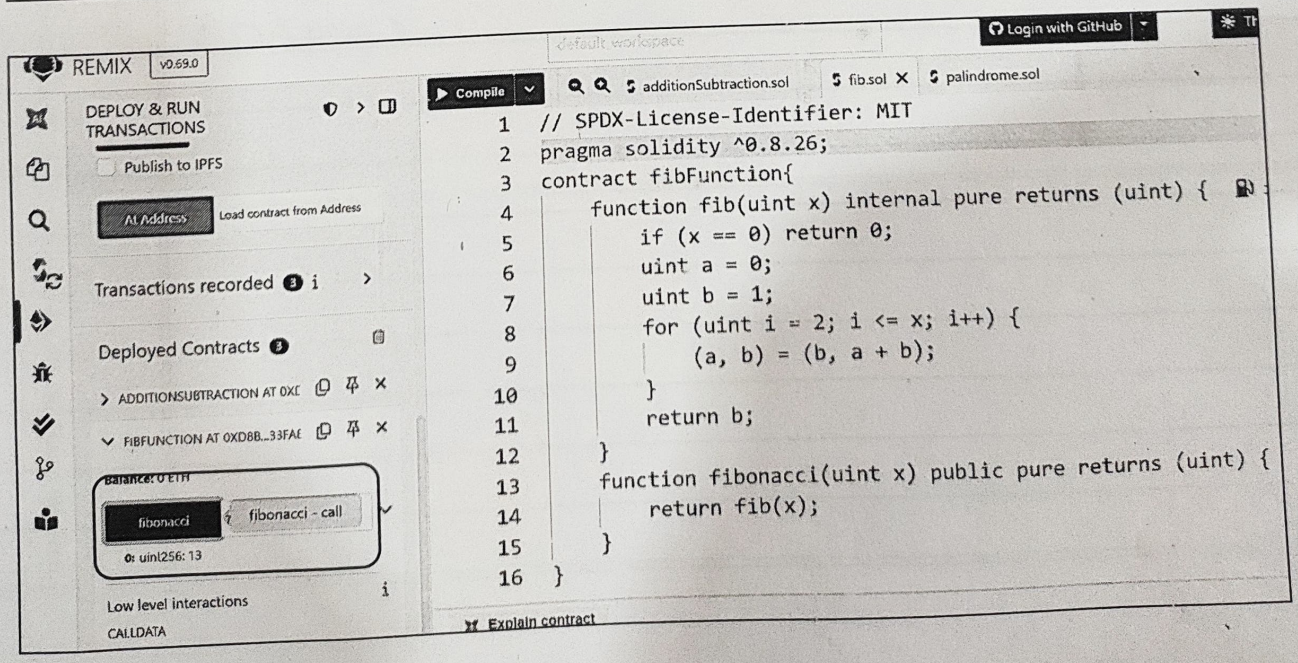
Program Code:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract fibFunction{
    function fib(uint x) internal pure returns (uint) {
        if (x == 0) return 0;
        uint a = 0;
        uint b = 1;
        for (uint i = 2; i <= x; i++) {
            (a, b) = (b, a + b); }
        return b;
    }
    function fibonacci(uint x) public pure returns (uint) {
        return fib(x);
    }
}
```

Output:





Experiment No-3

Creation of transaction in block chain

Solidity is a programming language used to write smart contracts on the Ethereum blockchain. Smart contracts are self-executing contracts that allow for the automatic transfer of digital assets when certain conditions are met. In this series, we will cover some of the more tricky areas of Solidity, aimed at the Intermediate level of Solidity skills.

In Part 3 we covered *Inheritance, Virtual, Override, and Super.*

This article will delve into the three primary methods for sending Ether: `transfer`, `send`, and `call`. We'll discuss their differences, use cases, and the recommended practices for advanced Solidity engineers.

Sending Ether: Transfer, Send, and Call

Solidity provides three methods to send Ether from one contract to another:

1. `transfer`: This method allows you to send Ether, consuming 2300 gas units. If the operation fails, it throws an error.

2. send: Similar to transfer, send also consumes 2300 gas units but returns a boolean value indicating the success or failure of the operation.
3. call: This method is more flexible, allowing you to forward all gas or set a specific gas limit. Like send, it returns a boolean value indicating the operation's success or failure.

When to Use Transfer, Send, or Call

The choice between transfer, send, and call depends on the specific requirements of your contract and the level of control you need over the transaction.

transfer: This was once the simplest and safest way to send Ether, as it automatically reverts the transaction if the call fails. However, it only forwards a fixed amount of gas (2300), which may not be enough for more complex operations in the receiving contract. This limitation has led to transfer being considered less safe, as it can lead to unexpected behavior if the receiving contract's fallback function consumes more than 2300 gas.

send: This method is similar to transfer but returns false instead of throwing an exception when the call fails. This allows you to handle the failure in your contract. However, like transfer , it also only forwards 2300 gas.

call: This is the most flexible method and is recommended for sending Ether as of Solidity 0.6.0. It allows you to forward all remaining gas or specify a certain

amount. This flexibility makes it suitable for more complex operations. However, it also requires you to handle the possibility of reentrancy attacks and manually check the return value.

Here's an example of how each method can be used in a contract:

```
// SPDX-License-Identifier: MIT

pragma solidity ^0.8.17;

contract SendEther {

  function sendViaTransfer(address payable _to) public payable {

    _to.transfer(msg.value); // Automatically reverts on failure

  }

  function sendViaSend(address payable _to) public payable {

    bool sent = _to.send(msg.value); // Returns false on failure
```

```
        require(sent, "Failed to send Ether");

    }

    function sendViaCall(address payable _to) public payable {

        (bool sent, bytes memory data) = _to.call{value: msg.value}(""); // Returns false on failure

        require(sent, "Failed to send Ether");

    }

}
```

Receiving Ether: Receive and Fallback Functions

A contract that is meant to receive Ether must implement at least one of the following functions:

1. `receive() external payable`: This function is triggered when the contract receives plain Ether (without data).
2. `fallback() external payable`: This function is triggered when the received message contains data or when no other function matches the function identifier.

The receive function was introduced in Solidity 0.6.0 as a new and more explicit way to mark a contract as being able to receive Ether. If a contract has a receive function, it will be called whenever a callto the contract is made with empty calldata. However, if there is data in the calldata, or if the contract does not have a receive function, the fallback function will be called. If the contract neither has a receive nor a fallback function, it cannot receive Ether through regular transactions and will throw an exception.

Here's a simple contract that can receive Ether:

```
// SPDX-License-Identifier: MIT

pragma solidity ^0.8.17;

contract ReceiveEther {

  receive() external payable {}

  fallback() external payable {}

  function getBalance() public view returns (uint) {
```

```
    return address(this).balance;

  }

}
```

Handling Errors: Revert, Assert, and Require

Solidity provides error-handling mechanisms like `revert`, `assert`, and `require` that you can use to handle errors in your contracts.

- `revert`: This function aborts execution and reverts state changes but consumes all remaining gas and provides an error message.
- `assert`: This function should only be used to test for internal errors and to check invariants. If the condition inside `assert` fails, it causes a runtime error, which leads to the transaction being reverted.
- `require`: This function should be used to ensure valid conditions, such as inputs, or contract state variables are met, or to validate return values from calls to external contracts.

In the context of sending Ether, you can use `require` to ensure that the `send` or `call` the operation was successful:

```
// SPDX-License-Identifier: MIT

pragma solidity ^0.8.17;

contract SendEther {

  function sendViaSend(address payable _to) public payable {

      bool sent = _to.send(msg.value);

      require(sent, "Failed to send Ether"); // Reverts transaction if send fails

  }

  function sendViaCall(address payable _to) public payable {

      (bool sent, bytes memory data) = _to.call{value: msg.value}("");

      require(sent, "Failed to send Ether"); // Reverts transaction if call fails

  }
```

```
}
```

Guarding Against Re-entrancy

Repentance is a common attack vector in smart contracts where an attacker can drain a contract of its Ether. To guard against re-entrance, you should:

- Make all state changes before calling other contracts.
- Use a re-entrance guard modifier.

As of December 2019, the recommended method for sending Ether is `call` in combination with a re-entrance guard.

Conclusion

Understanding the nuances of sending and receiving Ether is crucial for writing secure, efficient smart contracts in Solidity. While `transfer` and `send` were once the standard, the flexibility and safety of `call` have made it the recommended method for sending Ether as of recent Solidity versions. Always remember to guard against reentrancy attacks when writing your contracts. Happy coding!

What is MetaMask?

MetaMask is a blockchain (Cryptocurrency) wallet used for storing digital coins and money in online mode safely and securely. It provides a digital bank where you can keep your bitcoin and other cryptocurrencies.It is mainly used to interact with the Ethereum blockchain. It is fully equipped with safety and straightforward to use.In simple words, MetaMask is one of the globally known cryptocurrency wallets. It is a type of browser extension used with the help of web browsers. An extension is a specific type of application that makes interacting with the Ethereum network simpler using modern web browsers.MetaMask supplies its customers a software that allows them to manage Ether and other ERC 20 currencies and engage with Ethereum decentralized Apps.Its browser extension is use to transmit, buy, receive, and acquire Ethereum or Ethereum-compatible tokens.Public blockchains like Ethereum are the next evolution of Internet databases, and MetaMask is the next evolution of the browser. MetaMask keeps no information on you: not your email address, not your password, and not your Secret Recovery Phrase or other private keys. You retain all power over your crypto-identity.

How to install MetaMask:

- Browser extension
- Mobile app
- Chrome,Firefox,Edge,Brave
- Alternatively, you could also access each browser's extension store and search for MetaMask. Whatever you prefer!
- Configure your privacy settings

// When installing the MetaMask extension, you'll be able to adjust privacy settings one-by-one to ensure the app is set up according to your privacy preferences.

To do this, simply hit the 'Advanced configuration' button in the wallet set-up process.

Extension permissions

When adding MetaMask to your browser, you may ask why you 'd have to approve an extension to “read and change all your data on the websites you visit”. Get ready, the answer is technical:

In order to enable dapps (decentralized applications) to access the blockchain, MetaMask needs to inject a Web3 JavaScript object into each page. By doing so, it will not change the website, but merely allows it to access the network. For a fuller explanation of this, see here.

If you are still not convinced, a good way to experiment and manage your browser is to sandbox your MetaMask: create a separate browser profile so that MetaMask is only installed there. This will let you get used to MetaMask and Web3 in an environment that's separate from your existing web identity.

That said, MetaMask is safe for browsing. Welcome — you're joining over 30 million MetaMask users all over the world!

Fund your wallet

Adding some tokens to your wallet allows you to fully engage in web3. After creating your wallet, you'll see a prompt in the MetaMask extension to fund your wallet.

From there, you can buy crypto with a debit or credit card, receive crypto from another account, or transfer crypto from a CEX. Select the option you'd like to start with; you can always use a different method later.

Configure your privacy settings

When installing the MetaMask extension, you'll be able to adjust privacy settings one-by-one to ensure the app is set up according to your privacy preferences.

To do this, simply hit the 'Advanced configuration' button in the wallet set-up process. Here you can toggle/configure settings including:

Gas

Gas is the unit of measure for how much computational work is required to process transactions and smart contracts. Essentially a transaction fee, the term originates from Ethereum, in which context it refers to computation undertaken on the Ethereum Virtual Machine (EVM). Since Ethereum was founded, numerous EVM-compatible (and non-EVM-compatible!) networks have emerged and adopted similar models.

The term is analogous to the gas that powers a car engine: it's the fluctuating, occasionally

expensive cost of operation. More complex smart contracts require more gas to power their computation, just as a bigger, more powerful car takes more gas to run.

The method for calculating gas fees varies depending on the network. For example, calculating gas on Ethereum used to be very complicated, but was considerably simplified with the implementation of Ethereum Improvement Protocol (EIP) 1559 in August 2021 (also known as the London Upgrade). Essentially, you pay a base fee for every unit of gas, which is *burned* (read: it is deleted, and disappears) upon successful completion of the transaction. On top of the base fee, you add a priority fee, again per unit of gas, the value of which depends on how quickly you want the transaction to go through.

Across the broad range of EVM-compatible networks available, gas, or similarly-functioning alternatives, have essentially become the standard method of calculating transaction costs. Fees are paid in the network's native token: for example, any transaction on Ethereum requires ETH; using BSC requires BNB; using Polygon requires POL (previously MATIC). Some networks have adopted Ethereum's EIP-1559 model wholesale, such as Polygon, whilst others have made adjustments, including Avalanche, for their C-Chain (which burns both the base fee and priority fee, rather than just the former).

If you want to read a more in-depth look at how gas works on Ethereum, see here.

Here are some essential details for dealing with gas in MetaMask:

The gas limit (units of gas used)

The *gas limit* is the maximum number of units of gas you are willing to pay for in order to carry out a transaction or EVM operation. Different operations demand different quantities of gas units. A normal transaction sending ETH or a token normally costs 21,000 gas, whereas an ERC-20 token approval requires 45,000. Many networks, such as EVM-compatible blockchain Harmony, use an identical model in which standard transactions also cost 21,000 gas.

info

Do I need to edit gas limit?

No! MetaMask automatically sets your gas limit depending on the transaction you're trying to execute. In the vast majority of cases, this will be adequate to complete your transaction. If you

want to check or edit it, make sure you have advanced gas controls turned on and hit the button next to the gas information on the transaction confirmation screen that reads 'Market', 'Low', or 'Aggressive'.

The base fee

Every block on the Ethereum network has a base fee determined by network demand: the base fee is based on the block size of the block before it, compared against a target block size (where size refers to the total amount of gas used for all the transactions the block includes). If the size of the previous block exceeds the target, the base fee for the next block increases by 12.5%, leaving you, the user (or your wallet), with absolute certainty as to the base fee of the upcoming block. Your total gas fee must meet this price as a minimum in order to be considered for inclusion in the block.

The priority fee

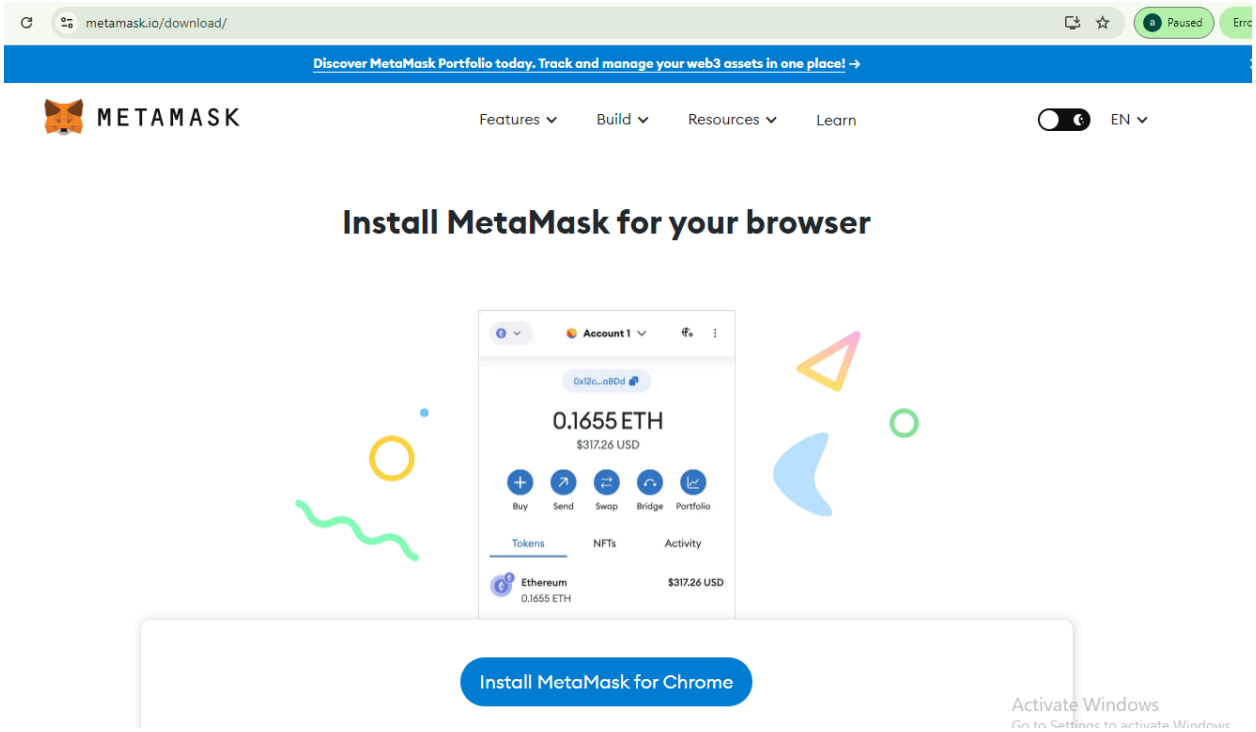
The *priority fee*, also referred to as the "miner tip", incentivizes the miner to prioritize your transaction.

Naturally, whether this does actually go to a miner depends on the consensus mechanism they use: Ethereum mainnet became a Proof of Stake network following the Merge in September 2022, so the priority fee goes to validators instead of miners.

The max fee

The max fee is the total, global amount paid for your transaction. It is calculated as: (base fee + priority fee) x units of gas used. MetaMask initially sets this amount based on the previous block's history. However, users can edit this amount through custom settings (see below). The difference between max fee per gas and (base fee + max priority fee per gas) is “refunded” to the user. /

Step1-


metamask.io/download/
Paused
Discover MetaMask Portfolio today. Track and manage your web3 assets in one place! →
METAMASK
Features
Build
Resources
Learn
EN
Install MetaMask for your browser
Account 1
0x12c...oBDd
0.1655 ETH
$317.26 USD
Buy
Send
Swap
Bridge
Portfolio
Tokens
NFTs
Activity
Ethereum
0.1655 ETH
$317.26 USD
Install MetaMask for Chrome
Activate Windows

Step2-

METAMASK

English

Help us improve MetaMask

We'd like to gather basic usage and diagnostics data to improve MetaMask. Know that we never sell the data you provide here.

Learn how we protect your privacy while collecting usage data for your profile.

When we gather metrics, it will always be...

- ✓ **Private:** clicks and views on the app are stored, but other details (like your public address) are not.
- ✓ **General:** we temporarily use your IP address to detect a general location (like your country or region), but it's never stored.
- ✓ **Optional:** you decide if you want to share or delete your usage data via settings any time.

☑ We'll use this data to learn how you interact with our marketing communications. We may share relevant news (like product features).

We'll let you know if we decide to use this data for other purposes. You can review our Privacy Policy for more information. Remember, you can go to settings and opt out at any time.

No thanks | I agree

Activate Windows

METAMASK

English

Let's get started

Trusted by millions, MetaMask is a secure wallet making the world of web3 accessible to all.

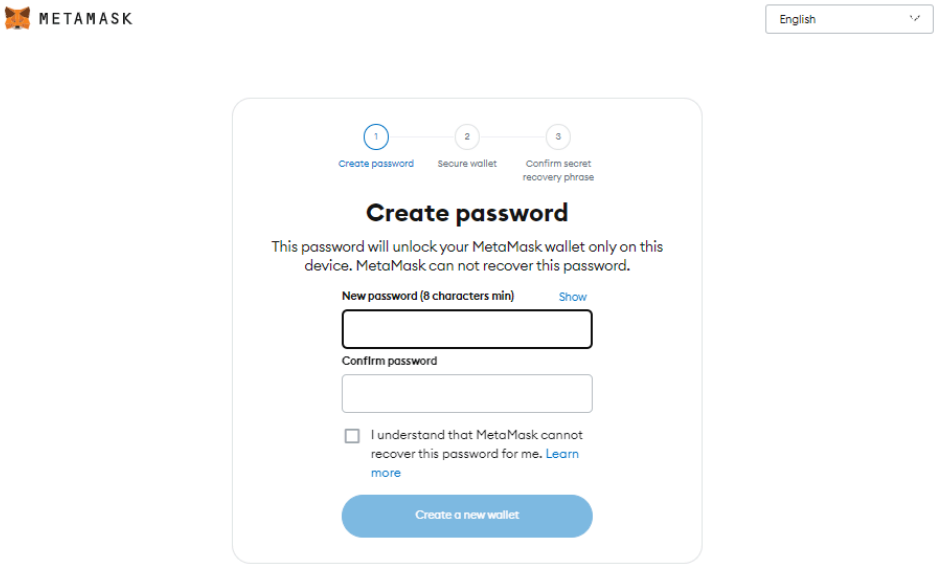


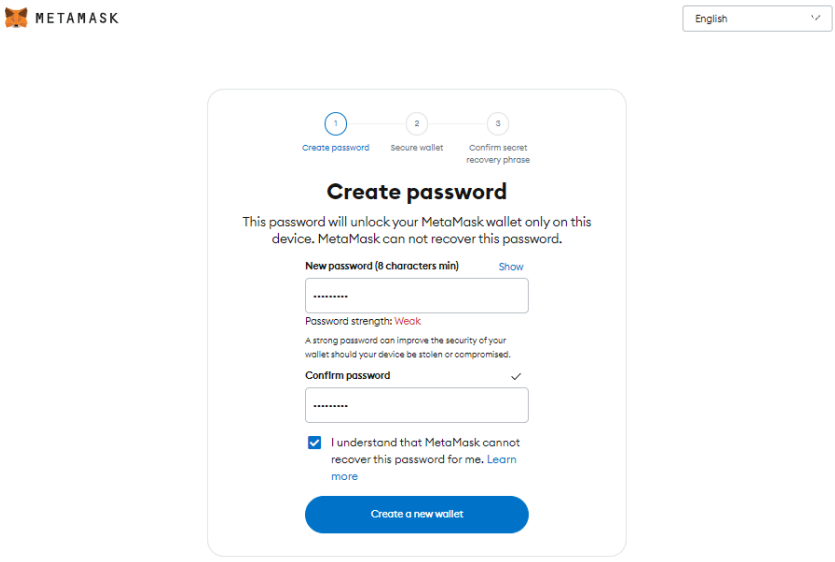
☑ I agree to MetaMask's Terms of use

Create a new wallet

Import an existing wallet

Activate Windows
Go to Settings to activate Win


METAMASK
English
1
Create password
2
Secure wallet
3
Confirm secret recovery phrase
Create password
This password will unlock your MetaMask wallet only on this device. MetaMask can not recover this password.
New password (8 characters min)
Show
Confirm password
I understand that MetaMask cannot recover this password for me. Learn more
Create a new wallet


METAMASK
English
1
Create password
2
Secure wallet
3
Confirm secret recovery phrase
Create password
This password will unlock your MetaMask wallet only on this device. MetaMask can not recover this password.
New password (8 characters min)
Show
Password strength: Weak
A strong password can improve the security of your wallet should your device be stolen or compromised.
Confirm password
I understand that MetaMask cannot recover this password for me. Learn more
Create a new wallet

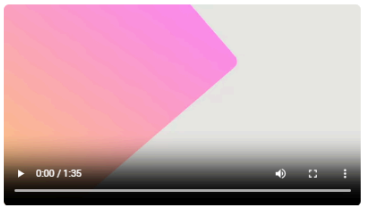
METAMASK

English

1 Create password — 2 Secure wallet — 3 Confirm secret recovery phrase

Secure your wallet

Before getting started, watch this short video to learn about your Secret Recovery Phrase and how to keep your wallet safe.



Remind me later (not recommended) | Secure my wallet (recommended)

What is a Secret Recovery Phrase?
Your Secret Recovery Phrase is a 12-word phrase that is the "master key" to your wallet and your funds

How do I save my Secret Recovery Phrase?
- Save in a password manager
- Store in a safe deposit box
- Write down and store in multiple secret places

Should I share my Secret Recovery Phrase?

Activate Windows
Go to Settings to activate Windows.

METAMASK

English

1 Create password — 2 Secure wallet — 3 Confirm secret recovery phrase

Write down your Secret Recovery Phrase

Write down this 12-word Secret Recovery Phrase and save it in a place that you trust and only you can access.

Tips:
- Save in a password manager
- Store in a safe deposit box
- Write down and store in multiple secret places

1. still	2. fan	3. must
4. toe	5. enact	6. mushroom
7. stand	8. tuition	9. biology
10. easily	11. coyote	12. unusual

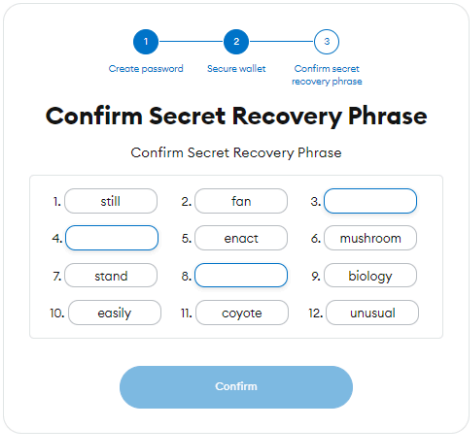
Hide seed phrase | Copy to clipboard

Next

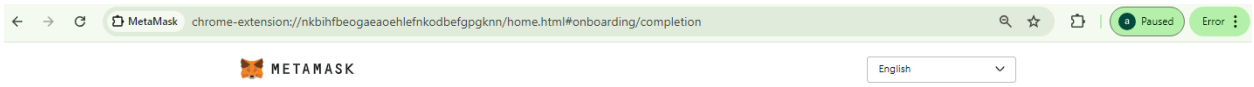
Activate Windows
Go to Settings to activate Wi

METAMASK

English



Activate Windows



METAMASK

English



Wallet creation successful

You've successfully protected your wallet. Keep your Secret Recovery Phrase safe and secret -- it's your responsibility!

Remember:

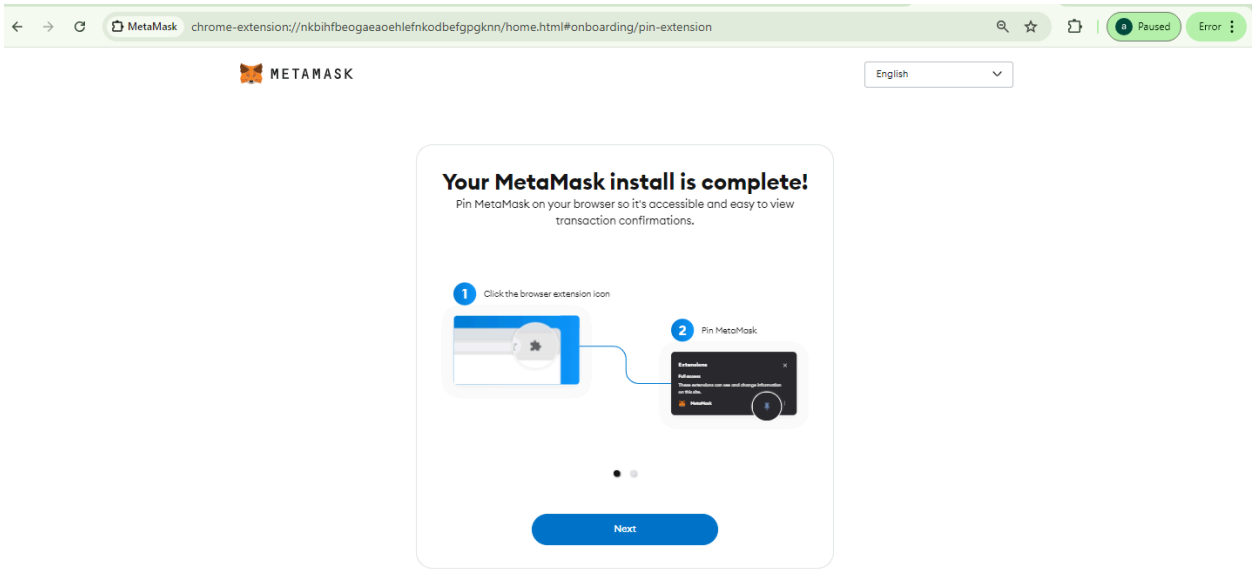
- MetaMask can't recover your Secret Recovery Phrase.
- MetaMask will never ask you for your Secret Recovery Phrase.
- **Never share your Secret Recovery Phrase** with anyone or risk your funds being stolen
- Learn more

Advanced configuration

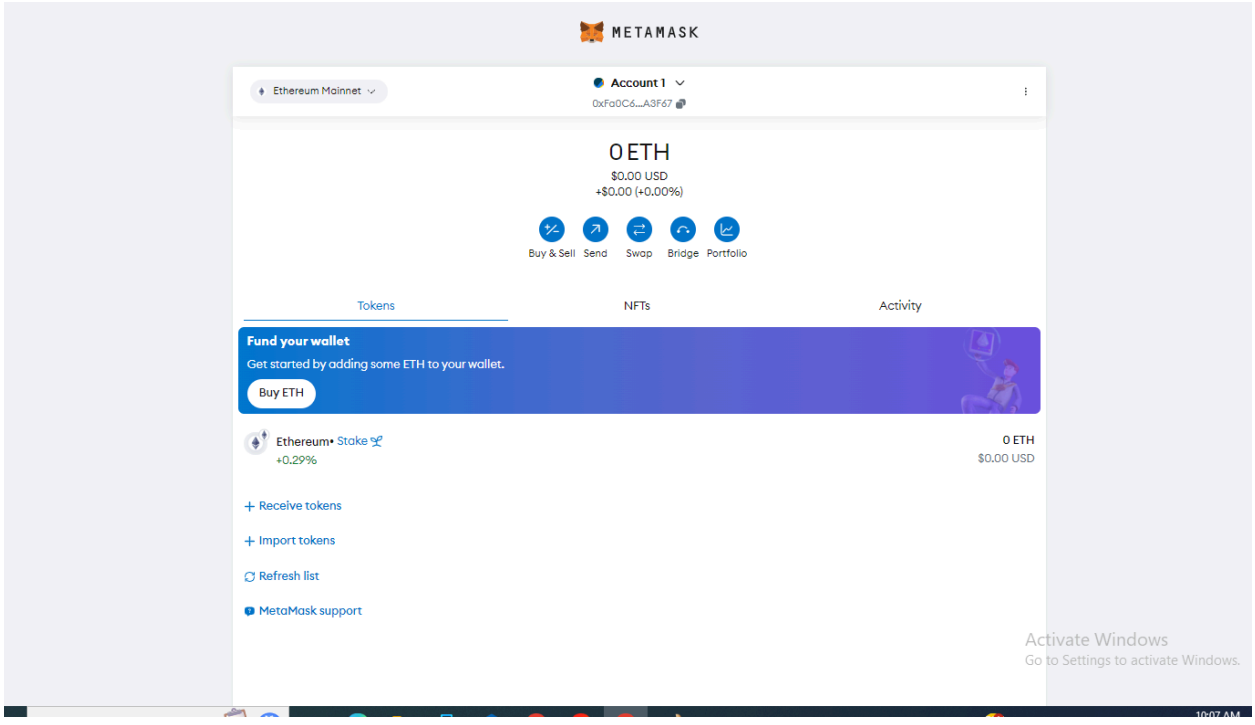
Got it

Follow us on Twitter

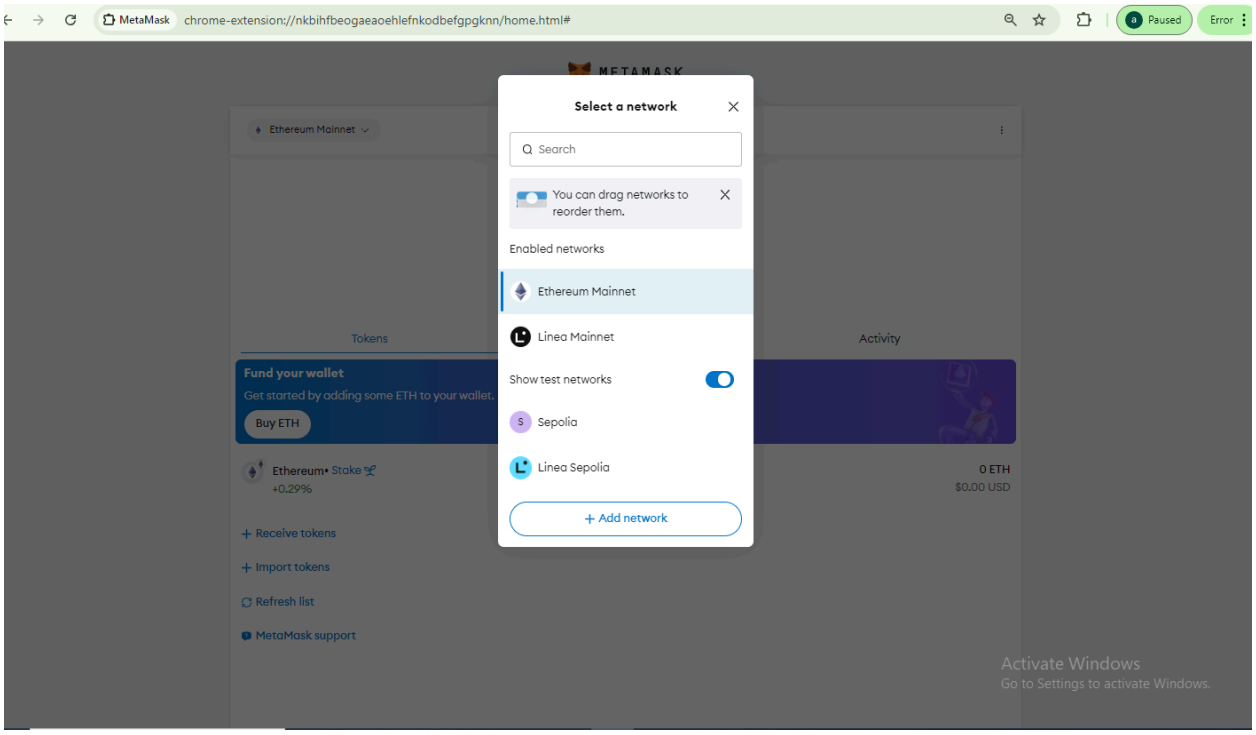
Activate Windows

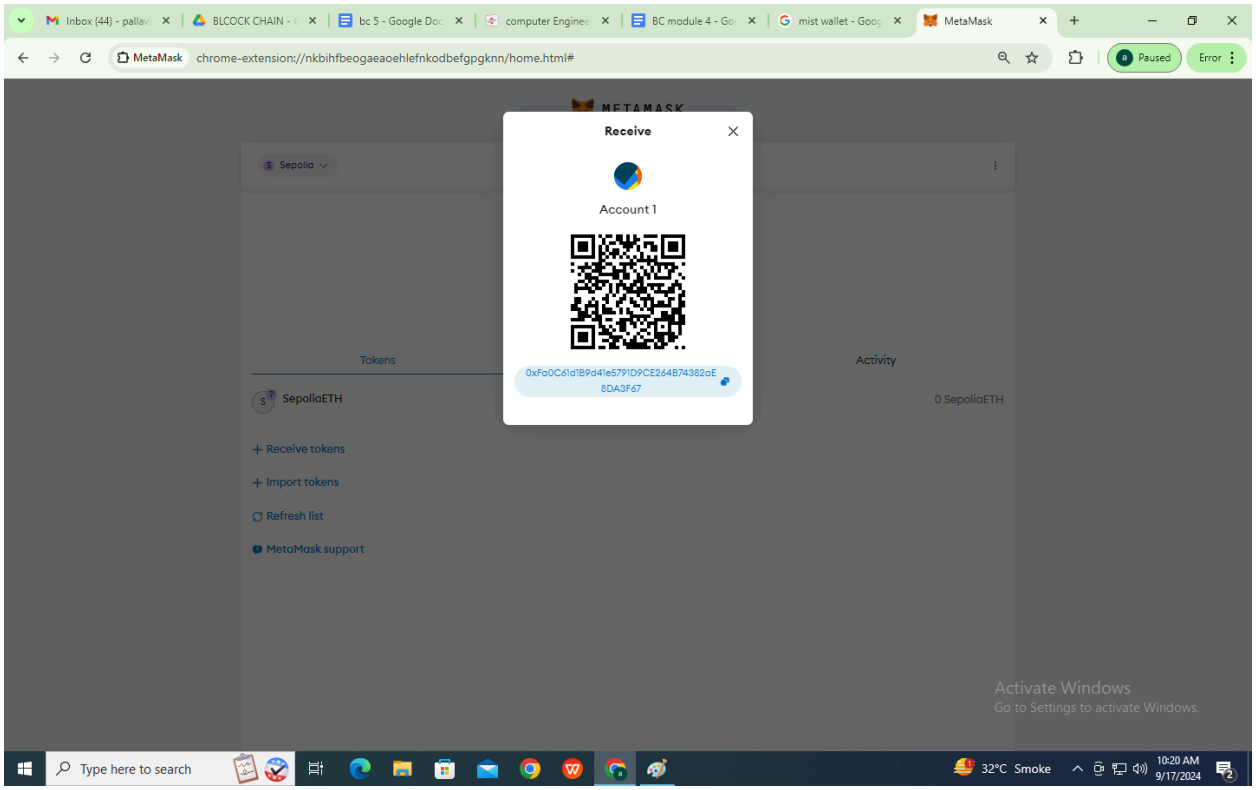


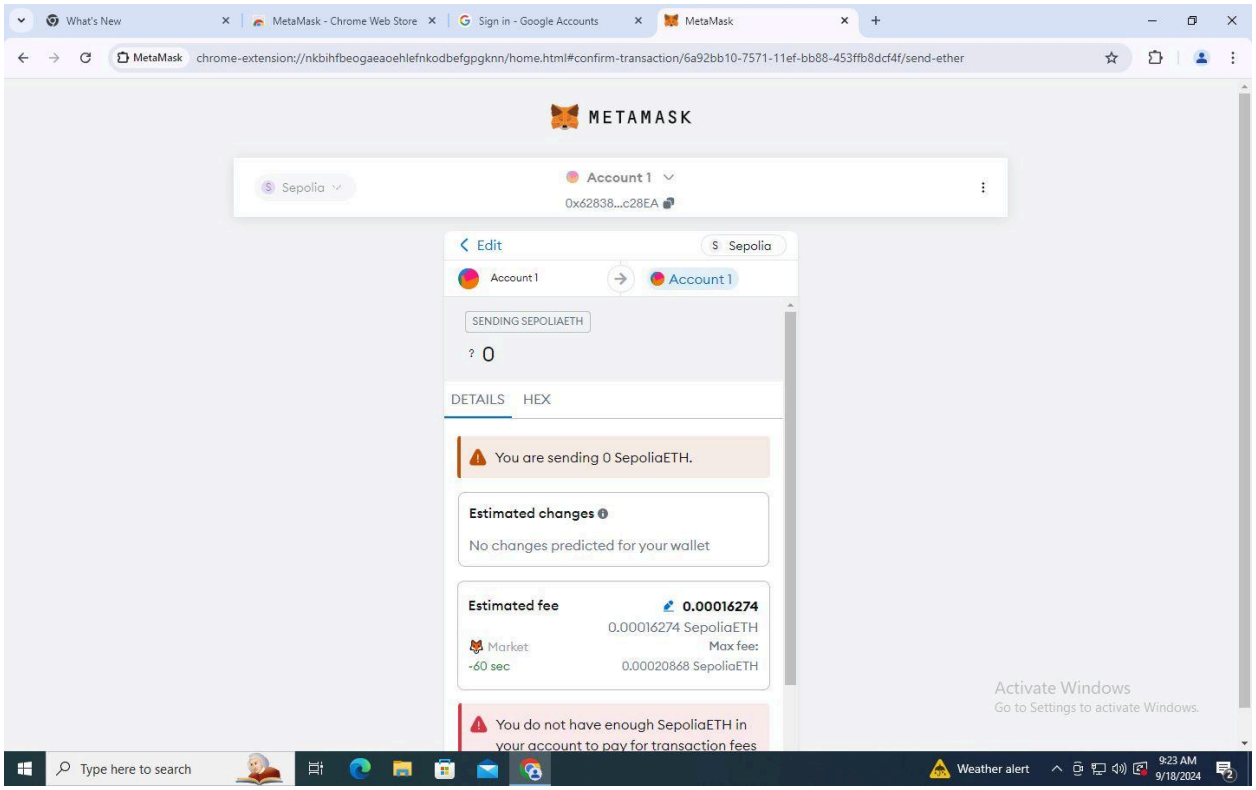
Activate Windows
Go to Settings to activate Windows.



Activate Windows
Go to Settings to activate Windows.


MetaMask
chrome-extension://nkbihfbeogaeaoehlefnkodbefgpgknn/home.html#
Paused
Error
METAMASK
Ethereum Mainnet
Tokens
Activity
Fund your wallet
Get started by adding some ETH to your wallet.
Buy ETH
Ethereum• Stake
+0.29%
0 ETH
$0.00 USD
+ Receive tokens
+ Import tokens
Refresh list
MetaMask support
Select a network
Search
You can drag networks to reorder them.
Enabled networks
Ethereum Mainnet
Linea Mainnet
Show test networks
Sepolia
Linea Sepolia
+ Add network
Activate Windows
Go to Settings to activate Windows.


MetaMask
chrome-extension://nkbihfbeogaeaoehlefnkodbefgpgknn/home.html#
Paused
Error
METAMASK
Sepolia
Receive
Account 1
0xFa0C61d1B9d41e5791D9CE264B74382aE8DA3F67
Tokens
Activity
SepoliaETH
0 SepoliaETH
Receive tokens
Import tokens
Refresh list
MetaMask support
Activate Windows
Go to Settings to activate Windows.
Type here to search
32°C Smoke
10:20 AM
9/17/2024


What's New
MetaMask - Chrome Web Store
Sign in - Google Accounts
MetaMask
chrome-extension://nkbihfbeogaeaoehlefnkodbefgpgknn/home.html#confirm-transaction/6a92bb10-7571-11ef-bb88-453ffb8dcf4f/send-ether
METAMASK
Sepolia
Account 1
0x62838...c28EA
Edit
Sepolia
Account 1
Account 1
SENDING SEPOLIAETH
? 0
DETAILS
HEX
You are sending 0 SepoliaETH.
Estimated changes
No changes predicted for your wallet
Estimated fee
0.00016274
0.00016274 SepoliaETH
Market
Max fee:
~60 sec
0.00020868 SepoliaETH
You do not have enough SepoliaETH in your account to pay for transaction fees
Activate Windows
Go to Settings to activate Windows.
Type here to search
Weather alert
9:23 AM
9/18/2024

Expt 5 : Blockchain platform Ganache.

Ethereum - Ganache for Blockchain

Ganache is used for setting up a personal Ethereum Blockchain for testing your Solidity contracts. It provides more features when compared to Remix.Ganache is a popular development tool provided by Truffle Suite for creating a personal Ethereum blockchain for development and testing purposes. It is designed to make blockchain development faster and easier by providing a fully controlled environment.

Step 1 : Download and install the Blockchain on your local machine.

Downloading Ganache

You may download Ganache from t

he following URL –

https://archive.trufflesuite.com/ganache/

Download for windows



Installing Ganache

Locate the "Ganache-2.0.0.dmg" in your Downloads folder and double-click on it to install Ganache. Upon successful installation, the following screen will appear –

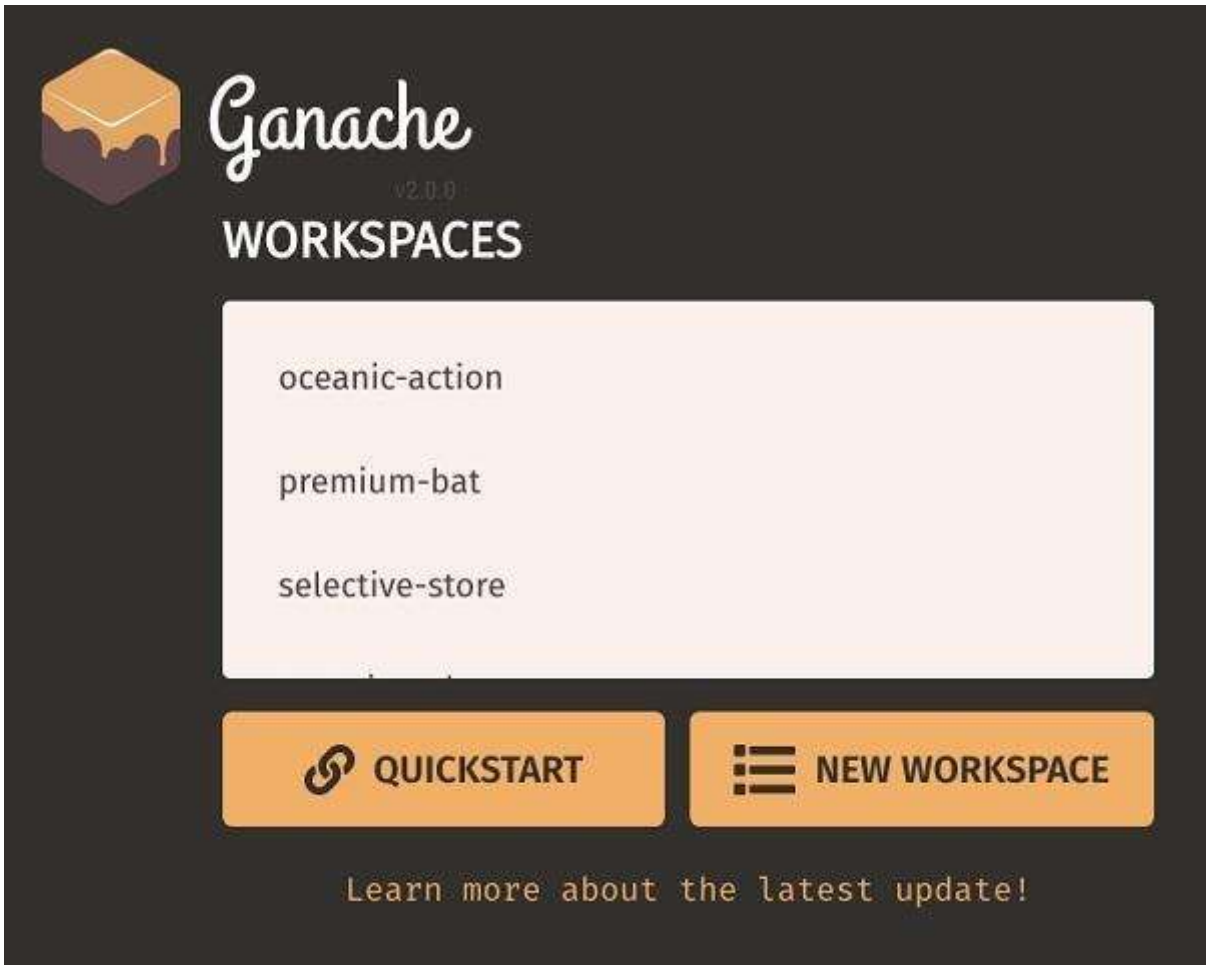


Starting Ganache

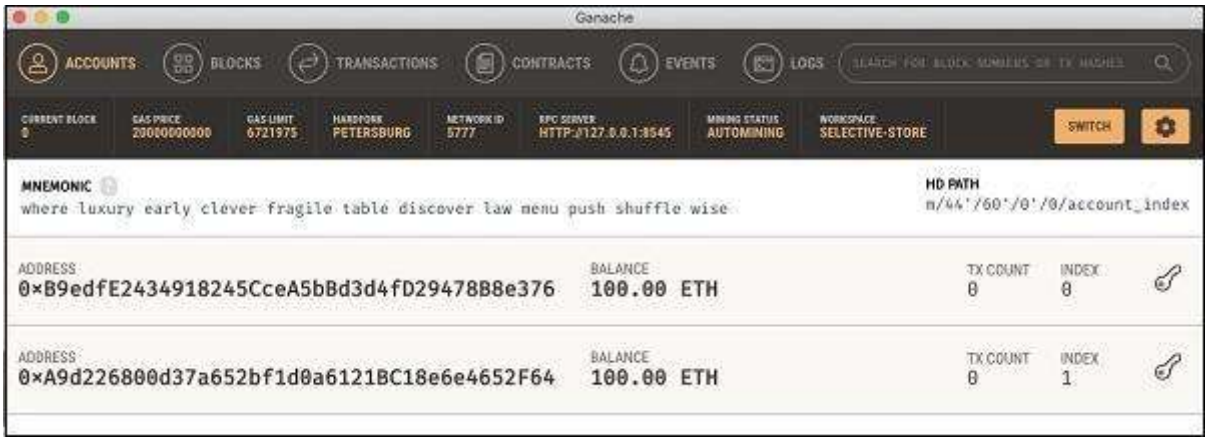
Now locate Ganache in your Application folder and double-click on its icon to start Ganache.

Ganache Desktop

When Ganache starts, the Ganache screen will appear as shown below –



Click QUICKSTART to start Ganache. You will see Ganache console as shown below –



The console in the above screenshot shows two user accounts with balance of 100 ETH (Ether - a currency for transaction on Ethereum platform). It also shows a transaction count of zero for each account. As the user has not performed any transactions so far, this count is obviously zero.

Conclusion - Thus we have successfully created the Blockchain platform Ganache.

Expt 5 : Blockchain platform Ganache.

Ethereum - Ganache for Blockchain

Ganache is used for setting up a personal Ethereum Blockchain for testing your Solidity contracts. It provides more features when compared to Remix.Ganache is a popular development tool provided by Truffle Suite for creating a personal Ethereum blockchain for development and testing purposes. It is designed to make blockchain development faster and easier by providing a fully controlled environment.

Step 1 : Download and install the Blockchain on your local machine.

Downloading Ganache

You may download Ganache from the following URL –

https://archive.trufflesuite.com/ganache/

Download for windows



Installing Ganache

Locate the "Ganache-2.0.0.dmg" in your Downloads folder and double-click on it to install Ganache. Upon successful installation, the following screen will appear –

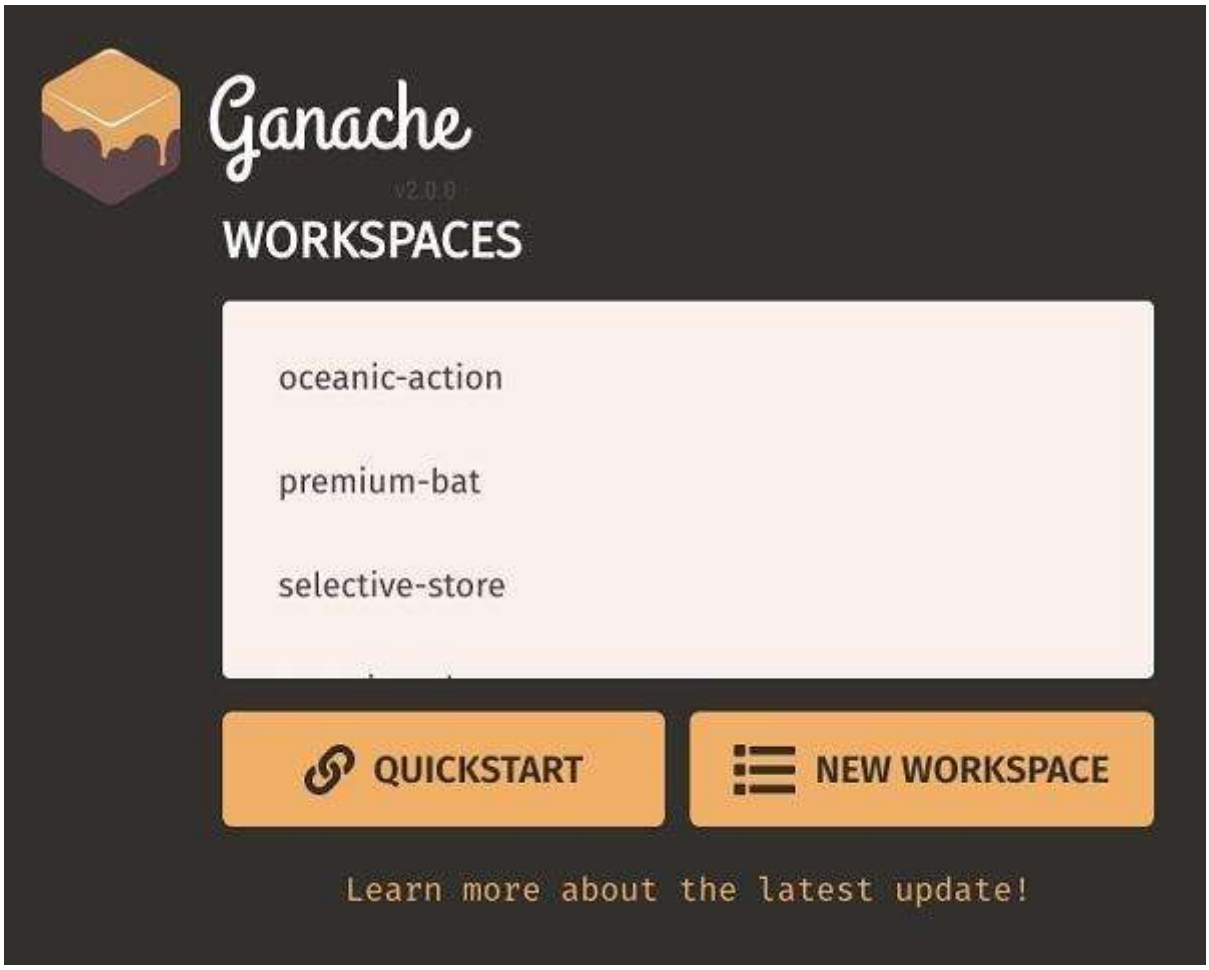


Starting Ganache

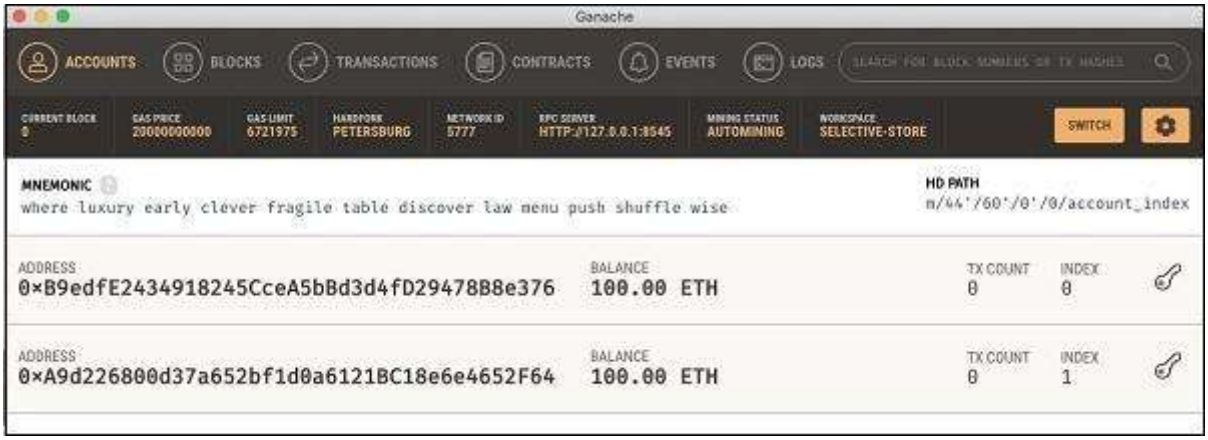
Now locate Ganache in your Application folder and double-click on its icon to start Ganache.

Ganache Desktop

When Ganache starts, the Ganache screen will appear as shown below –



Click QUICKSTART to start Ganache. You will see Ganache console as shown below –



The console in the above screenshot shows two user accounts with balance of 100 ETH (Ether - a currency for transaction on Ethereum platform). It also shows a transaction count of zero for each account. As the user has not performed any transactions so far, this count is obviously zero.

Conclusion - Thus we have successfully created the Blockchain platform Ganache.

Experiment No-6

Blockchain platform ethereum using Geth.

What is Blockchain?

- Blockchain is a chain of blocks that contains all information about all the transactions that take place in a network in an encrypted form by using public and private keys.
- Blockchain technology can strengthen the basic services that are essential in trade finance. The blockchain model works on a decentralized, digitalized, and distributed ledger model. Because of these properties, this is more robust and secure than the proprietary, centralized which are currently used in the trading system.
- In the simplest of terms, a blockchain is just a new form of a decentralized database.

What is Ethereum?

Ethereum is a decentralized open-source blockchain system that features its own cryptocurrency which is called ether(ETH). It is a platform that can be used for various apps which can be deployed by using Smart Contracts.

Why Build a Private Blockchain Network?

- In an Ethereum network if the nodes are not connected to the main network then it is known as a Private Ethereum Network.
- Only the nodes with the right permissions will be able to access this Blockchain.

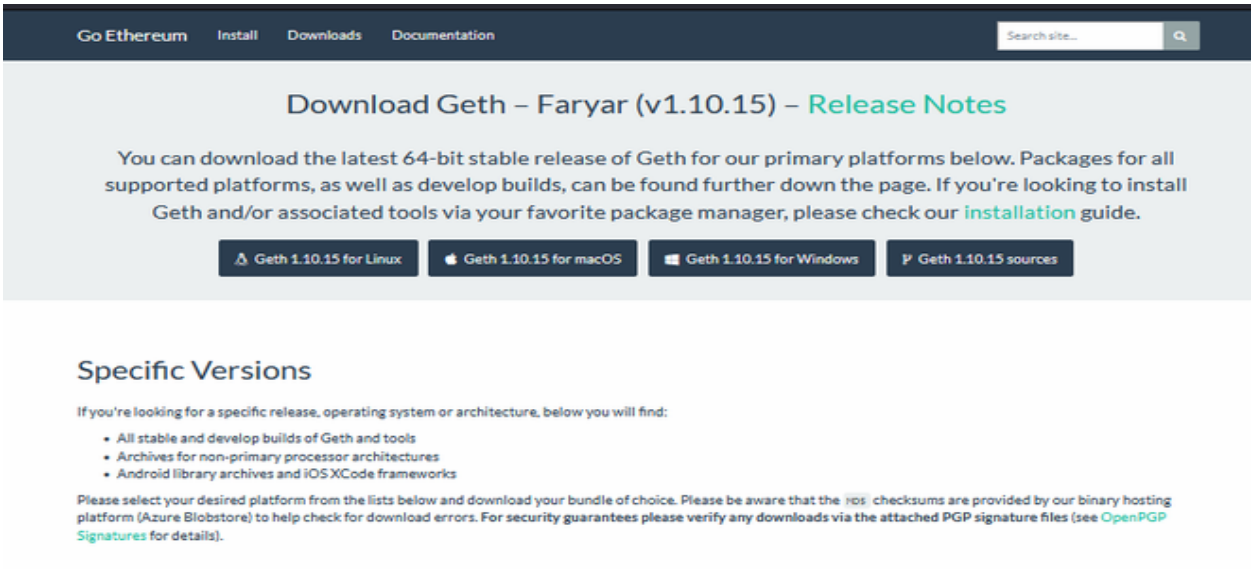
- Privacy – Don’t want to key data on a public network that’s why enterprises build a private network.
- To test the smart contracts and to develop smart contracts.

Steps to Set Up Private Ethereum Network

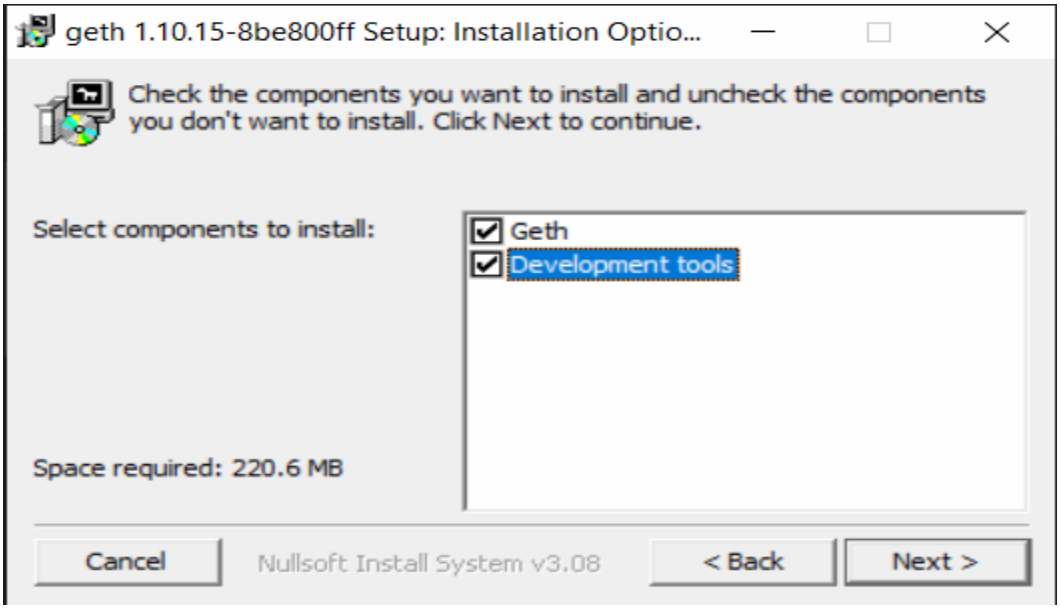
Below is the step-by-step guide to setting up a private Ethereum network.

Step 1: Install Geth on Your System

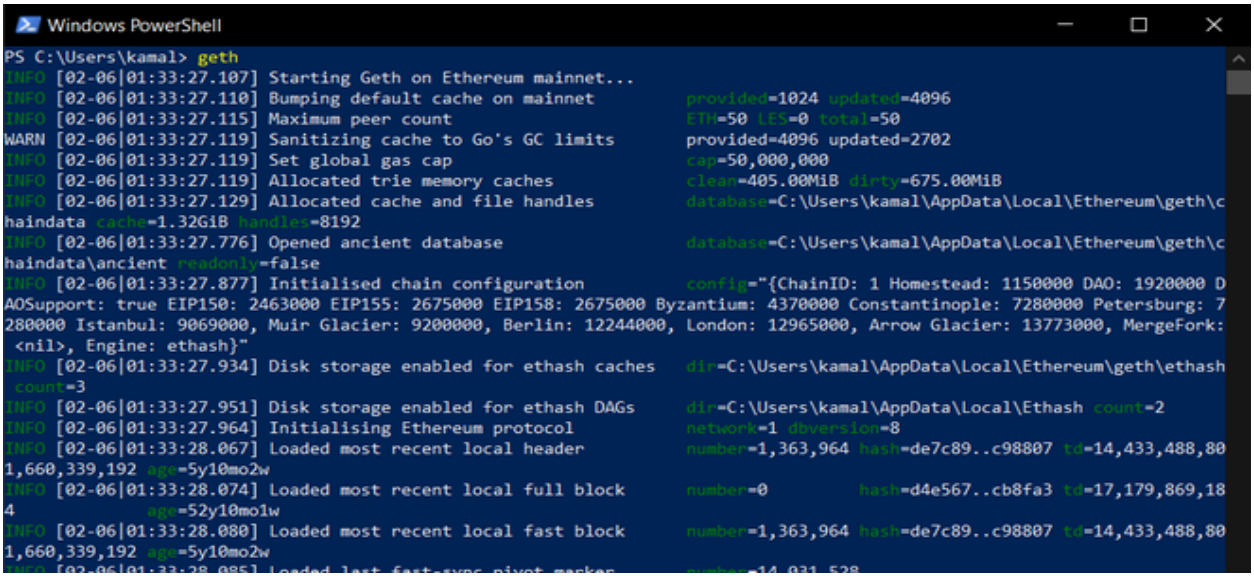
- Click here to Go to the official Geth download page and download setup according to your operating system.



- While installing Geth make sure to select both checkboxes as shown below.



After installing Geth on your system open PowerShell or command prompt and type geth and press enter, the following output will be displayed.

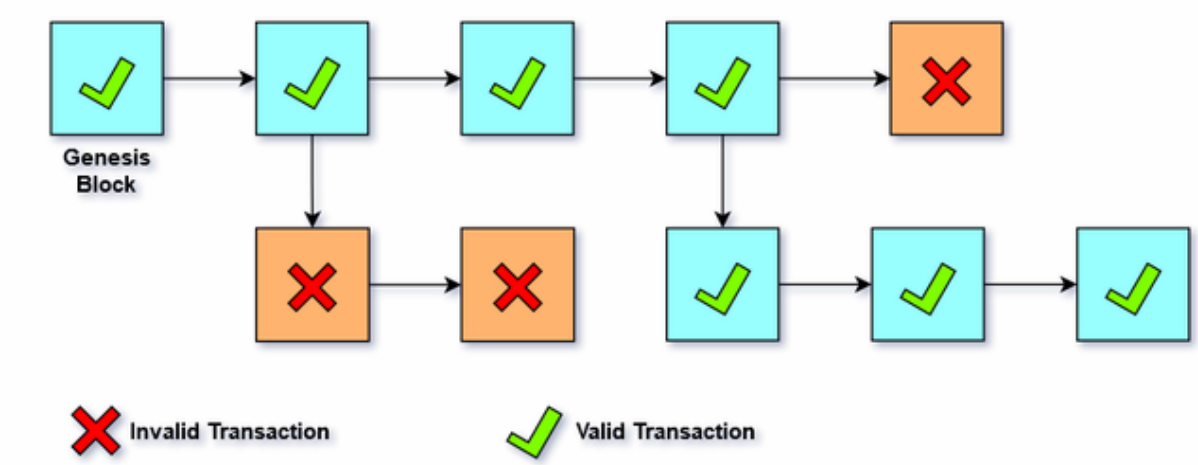


Step 2: Create a Folder For Private Ethereum

- Create a separate folder for this project. In this case, the folder is **MyNetwork.**
- Create a new folder inside the folder MyNetwork for the private Ethereum network as it keeps your Ethereum private network files separate from the public files. In this example folder is **MyPrivateChain.**

Step 3: Create a Genesis Block

The blockchain is a distributed digital register in which all transactions are recorded in sequential order in the form of blocks. There are a limitless number of blocks, but there is always one separate block that gave rise to the whole chain i.e. the genesis block.



As seen in the above diagram we can see that blockchain is initialized with the genesis block.

To create a private blockchain, a genesis blockis needed. To do this, create a genesis file, which is a JSON file with the following commands-

{

```
    “config”:{
      “chainId”:987,
      “homesteadBlock”:0,
      “eip150Block”:0,
      “eip155Block”:0,
      “eip158Block”:0
   },
   “difficulty”:”0x400",
   “gasLimit”:”0x8000000",
   “alloc”:{}
}
```

Explanation:

- **config:** It defines the blockchain configuration and determines how the network will work.
- **chainId:** This is the chain number used by several blockchains. The Ethereum main chain number is “1”. Any random number can be used, provided that it does not match with another blockchain number.
- **homesteadBlock:** It is the first official stable version of the Ethereum protocol and its attribute value is “0”.
- One can connect other protocols such as Byzantium, eip155B, and eip158. To do this, under the homesteadBlock add the protocol name with the Block prefix (for example, eip158Block) and set the parameter “0” to them.
- **difficulty:** It determines the difficulty of generating blocks. Set it low to keep the complexity low and to avoid waiting during tests.
- **gasLimit:** Gas is the “fuel” that is used to pay transaction fees on the Ethereum network. The more gas a user is willing to spend, the higher will be the priority of his transaction in the queue. It is recommended to set this value to a high enough level to avoid limitations during tests.
- **alloc:** It is used to create a cryptocurrency wallet for our private blockchain and fill it with fake ether. In this case, this option will not be used to show how to initiate mining on a private blockchain.

This file can be created by using any text editor and save the file with **JSON extension** in the folder MyNetwork.

Step 4: Execute genesis file

Open cmd or PowerShell in admin mode enter the following command-

geth –identity “yourIdentity” init \path_to_folder\CustomGenesis.json –datadir

\path_to_data_directory\MyPrivateChain

Parameters-

path_to_folder- *Location of Genesis file.*

path_to_data_directory- *Location of the folder in which the data of our private chain will be stored.*

The above command instructs Geth to use the CustomGenesis.json file.

After executing the above command Geth is connected to the Genesis file and it seems like this:

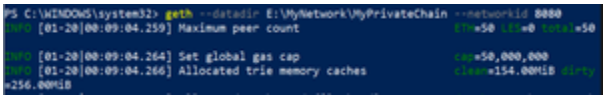
```
PS C:\WINDOWS\system32> geth --identity "MyBlockChain" init E:\MyNetwork\CustomGenesis.j
son --datadir E:\MyNetwork\MyPrivateChain
INFO [01-19|23:43:15.534] Maximum peer count                       ETH=50 LES=0 total=50

INFO [01-19|23:43:15.564] Set global gas cap                       cap=50,000,000
INFO [01-19|23:43:15.567] Allocated cache and file handles         database=E:\MyNetwork
\MyPrivateChain\geth\chaindata cache=16.00MiB handles=16
INFO [01-19|23:43:15.803] Writing custom genesis block
INFO [01-19|23:43:15.815] Persisted trie from memory database      nodes=0 size=0.00B ti
me="516µs" gcnodes=0 gcsize=0.00B gctime=0s livenodes=1 livesize=0.00B
INFO [01-19|23:43:15.846] Successfully wrote genesis state         database=chaindata ha
sh=d1a12d..4c8725
INFO [01-19|23:43:15.862] Allocated cache and file handles         database=E:\MyNetwork
\MyPrivateChain\geth\lightchaindata cache=16.00MiB handles=16
INFO [01-19|23:43:16.092] Writing custom genesis block
INFO [01-19|23:43:16.103] Persisted trie from memory database      nodes=0 size=0.00B ti
me=0s      gcnodes=0 gcsize=0.00B gctime=0s livenodes=1 livesize=0.00B
INFO [01-19|23:43:16.120] Successfully wrote genesis state         database=lightchainda
ta hash=d1a12d..4c8725
PS C:\WINDOWS\system32>
```

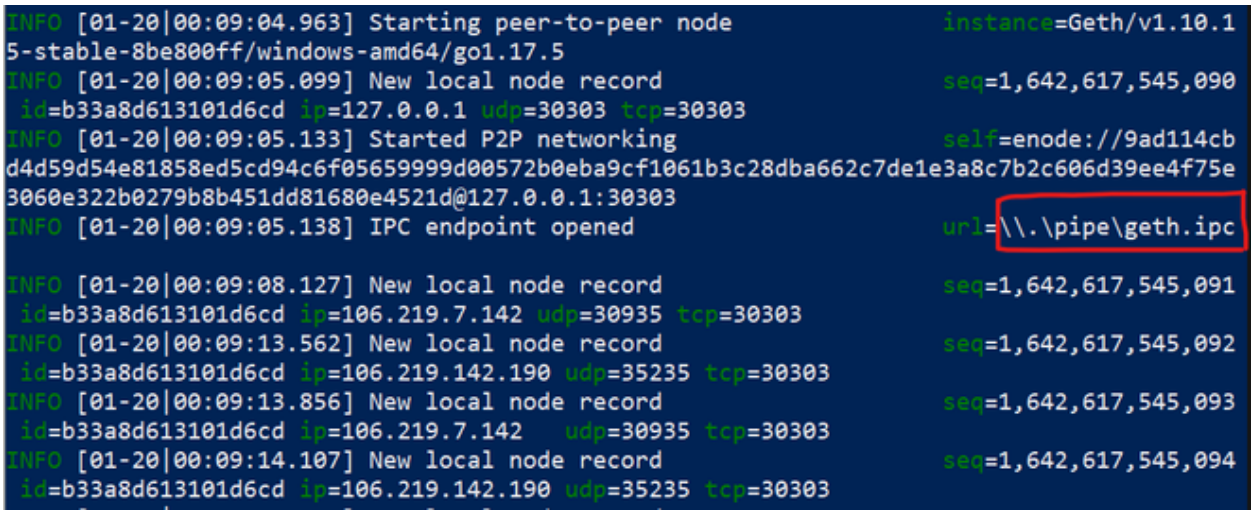
Step 5: Initialize the private network

Launch the private network in which various nodes can add new blocks for this we have to run the command-

geth –datadir \path_to_your_data_directory\MyPrivateChain –networkid 8080



The command also has the identifier **8080**. It should be replaced with an arbitrary number that is not equal to the identifier of the networks already created, for example, the identifier of the main network Ethereum ("networkid = 1"). After successfully executing the command we can see like this-



Note:

The highlighted text is the address of geth.ipc file finds it in your console and copy it for use in the next step.

Every time there is a need to access the private network chain, one will need to run commands in the console that initiate a connection to the Genesis file and the private network.

Now a personal blockchain and a private Ethereum network is ready.

Step 6: Create an Externally owned account(EOA)

Externally Owned Account(EOA) has the following features-

- Controlled by an External party or person.
- Accessed through private Keys.
- Contains Ether Balance.
- Can send transactions as well as 'trigger' contract accounts.

Steps to create EOA are:

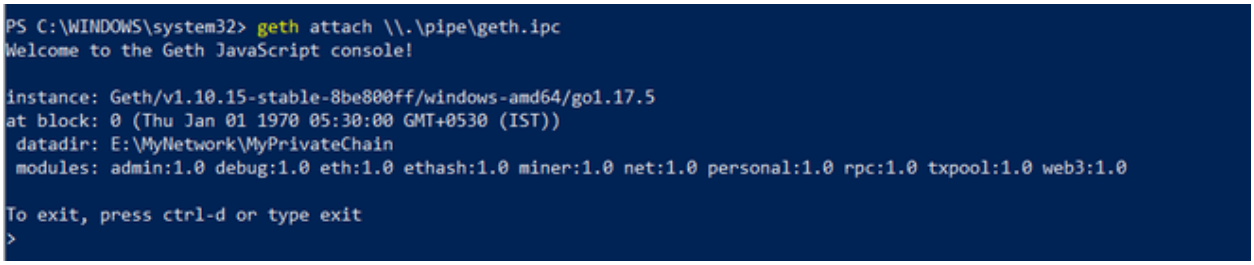
To manage the blockchain network, one need EOA. To create it, run Geth in two windows. In the second window console enter the following command-

geth attach \path_to_your_data_directory\YOUR_FOLDER\geth.ipc

or

geth attach \\.\pipe\geth.ipc

This will connect the second window to the terminal of the first window. The terminal will display the following-



Create an account by using the command-

personal.newAccount()

After executing this command enter Passphrase and you will get your account number and save this number for future use.

```
> personal.newAccount()
Passphrase:
Repeat passphrase:
"0x125c7bce5af112d0e271092be64c87ce5c31696c"
>
```

To check the balance status of the account execute the following command-

```
> eth.getBalance("0x125c7bce5af112d0e271092be64c87ce5c31696c")
0
```

It can be seen from the above screenshot that it shows zero balance. This is because when starting a private network in the genesis file, we did not specify anything in the alloc attribute.

Step 7: Mining our private chain of Ethereum

If we mine in the main chain of Ethereum it will require expensive equipment with powerful graphics processors. Usually, ASICs are used for this but in our chain high performance is not required and we can start mining by using the following command-

miner.start()

```
> miner.start()
null
```

If the balance status is checked after a couple of seconds the account is replenished with fake ether. After that, one can stop mining by using the following command-

miner.stop()

```
> eth.getBalance("0x125c7bce5af112d0e271092be64c87ce5c31696c")
200000000000000000000
> miner.stop()
null
>
```

Summer-time is here and so is the time to skill-up! More than 5,000 learners have now completed their journey from **basics of DSA to advanced level development programs** such as Full-Stack, Backend Development, Data Science.

Conclusion

Thus we have studied how to create ,initialize a private network, and manage ethereum blockchain Using Geth accounts in blockchain.

7 Case Study on Hyperledger

Hyperledger Fabric "Building Your First Network"

Hyperledger Fabric is a Blockchain framework that works as the foundation for developing applications with a modular architecture. It is an open-source engine for blockchain and takes care of the most important features for evaluating and using blockchain for business use cases. Hyperledger comes under an open-source Umbrella Project under the Linux Foundation that helps in different fields of finance, banking, IoT, supply chain, manufacturing, and technology.

Prerequisites for Hyperledger Fabric-Samples

In order to start the first Hyperledger fabric network Fabric-samples repository will be used.

This repository is a set of CLI tool binaries and Fabric Docker .

Install the below dependencies before starting the project:

1. Install Docker

Installation on Windows: Visit the link to install the Docker setup on Windows. Hyperledger Fabric samples being used here are highly dependent on the bash script. So in windows where you need to install docker, it is recommended to use WSL2(Windows Subsystem for LinuxFor WSL2 you need to ensure that the Docker

desktop has enabled Integration so that WSL2 can Properly interact with the docker elements. you can do this by going into settings in docker desktop GUI and under the Resources tab you need to find wsl integration and ensure the checkbox for enable integration is checked.

Note:

In case if you are using wsl2 in windows you need to mount your local disk or alternatively you can use mnt command to temporarily mount your local disk at a time.

Installation on Linux: Install the latest version of Docker if it is not already installed. Run the below command to install:

sudo apt-get -y install docker-compose

Installation on Mac: For macOS, it is recommended to use Homebrew to manage the prereqs.

/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/master/install.sh)"

The Xcode command line tools will be installed as part of the Homebrew installation. Once Homebrew is ready, installing the necessary prerequisites is very easy:

Homebrew v3.x:

brew install –cask –appdir="/Applications" docker

Docker Desktop must be launched to complete the installation so be sure to open the application after installing it:

open /Applications/Docker.app

2. Install Golang-go

Install the latest version of Go if it is not already installed (only required if writing Go chaincode or SDK applications).

Installation on Linux/Windows-wsl2:

sudo apt install golang-go

Installation on Mac:

brew install go@1.18.7

3. Install jq

Install the latest version of jq if it is not already installed (only required for channel configuration transactions).

Installation on Linux/Windows-wsl2:

sudo apt install jq

Installation on Mac:

brew install jq

4. Install Node/java

You can Install Node/java or use typescript to run the default queries present in fabric samples.

Installation on Linux/Windows-wsl2:

You can install the node in Linux/wsl2 using the below command:

npm install node

If needed for Linux you can install npm using the below command:

sudo apt install npm

Installation on Mac:

Install Node on Mac with the below command (npm will be installed with Node):

brew install node

5. Installing Fabric-samples Repository

To start out with fabric samples install the Fabric-samples bash script:

curl -sSLO https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install-fabric.sh && chmod +x install-fabric.sh

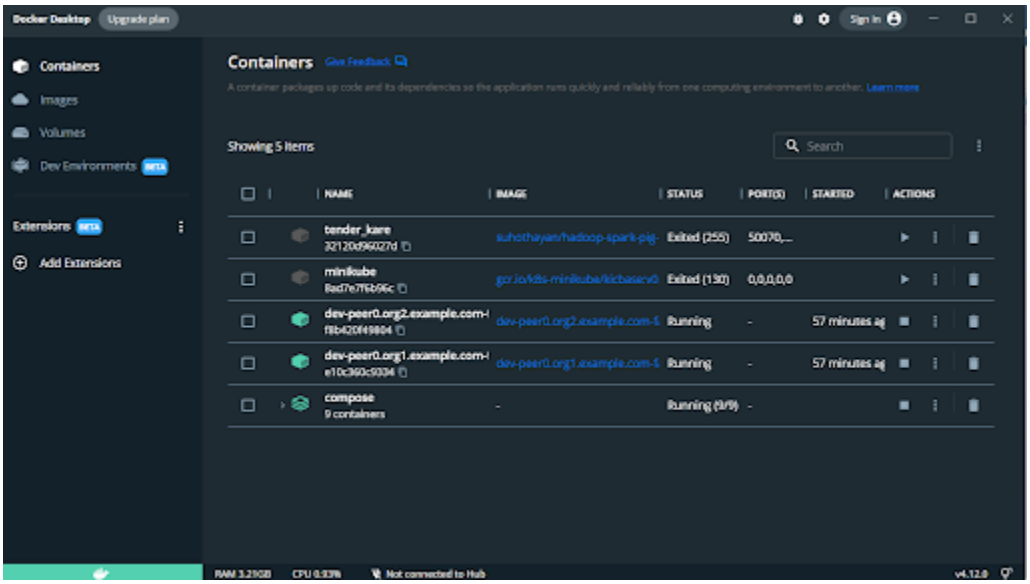
Then you can pull docker containers by running one of these commands:

./install-fabric.sh docker samples

./install-fabric.sh d s

To install binaries for Fabric samples you can use the command below:

./install-fabric.sh binary



After that installing all the above-required binaries and docker-compose containers you can check the compose containers either through docker desktop or in the case of Linux you can use the docker ps -a command to check all available containers and their respective status.

Building First Network

Fabric samples by default provide a Test Network that we can use to run our first network. Follow the steps below to build the first network:

Step 1: Navigate through the Fabric-samples folder and then through the Test network folder where you can find script files using these we can run our network. The test network is provided for learning about Fabric by running nodes on your local machine. Developers can use the network to test their smart contracts and applications. It is recommended not to change the script file.

cd fabric-samples/test-network

Step 2: From inside this directory, you can directly run ./network.sh script file through which we run the Test Network. By default, we would be running ./network.sh down command to remove any previous network containers or artifacts that still exist. This is to ensure that there are no conflicts when we run a new network.

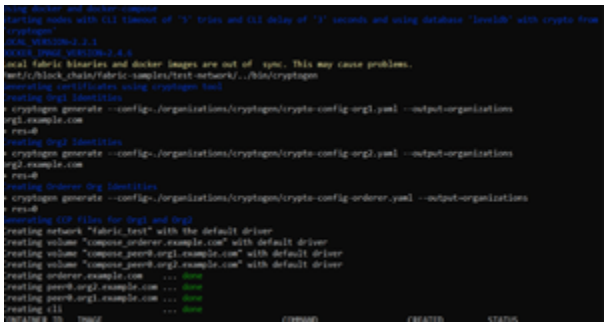
./network.sh down



Step 3: Now you can bring up a network using the following command. This command creates a fabric network that consists of two peer nodes and one ordering node. No channel is created when you run ./network.sh up

./network.sh up

If the above command completes successfully, you will see the logs of the nodes being created below picture:



Congratulation you have successfully run your first Hyperledger Fabric Network.

Summer-time is here and so is the time to skill-up! More than 5,000 learners have now completed their journey from **basics of DSA to advanced level development programs** such as Full-Stack, Backend Development, Data Science.

Sample Case Study on Hyperledger

Hyperledger is a collaborative project hosted by the Linux Foundation that focuses on developing open-source frameworks for enterprise blockchain solutions. It includes several distinct projects, each designed for different types of applications and use cases. Here's a case study overview of Hyperledger's key projects, illustrating how they have been used in various industries:

1. Hyperledger Fabric: A Supply Chain Management Solution

Industry: Supply Chain

Use Case: Walmart

Overview: Walmart, one of the world's largest retailers, has been using Hyperledger Fabric to enhance the traceability and transparency of its food supply chain. The project aims to improve food safety by tracking the provenance of products from farm to shelf.

Implementation:

- **Blockchain Network:** Hyperledger Fabric was chosen for its modularity and scalability.
- **Data Tracking:** The solution records every transaction and movement of products across the supply chain on the blockchain.
- **Verification:** The system enables quick tracing of products to their source, allowing Walmart to address potential issues rapidly and improve overall food safety.

Results:

- **Reduced Traceability Time:** Tracking a product's journey from farm to shelf, which previously took days, now takes seconds.
- **Enhanced Food Safety:** The ability to quickly identify and isolate contaminated products has significantly improved food safety standards.

2. Hyperledger Sawtooth: Automotive Industry

Industry: Automotive

Use Case: IBM's Automotive Blockchain

Overview: IBM has leveraged Hyperledger Sawtooth to develop a blockchain solution for the automotive industry, specifically to address issues related to vehicle lifecycle

management and the provenance of parts.

Implementation:

- **Blockchain Network:** IBM used Hyperledger Sawtooth's support for modular consensus and smart contracts to create a blockchain network for automotive parts and vehicle histories.
- **Data Management:** The blockchain records detailed histories of vehicle parts, including manufacturing details, ownership changes, and maintenance records.

Results:

- **Improved Transparency:** The blockchain solution provides a transparent and immutable record of vehicle and part histories, which helps in reducing fraud and ensuring quality.
- **Enhanced Efficiency:** Streamlined processes related to parts procurement, maintenance, and resale.

3. Hyperledger Indy: Digital Identity Management

Industry: Digital Identity

Use Case: Evernym's Self-Sovereign Identity

Overview: Evernym, a provider of decentralized identity solutions, utilizes Hyperledger Indy to build a self-sovereign identity system. This solution aims to give individuals control over their digital identities and personal data.

Implementation:

- **Blockchain Network:** Hyperledger Indy, designed specifically for decentralized identity management, was used to create a secure and privacy-preserving identity framework.
- **Identity Control:** Individuals can create and manage their digital identities, share credentials with third parties, and control access to their personal information.

Results:

- **User Empowerment:** Users have greater control over their identity and personal data, reducing reliance on central authorities.
- **Enhanced Privacy:** The system ensures that personal data is shared only with user consent and in a secure manner.

4. Hyperledger Iroha: Financial Services

Industry: Financial Services

Use Case: Soramitsu's Digital Asset Platform

Overview: Soramitsu, a Japanese fintech company, has used Hyperledger Iroha to develop a digital asset management platform. The platform is aimed at improving the efficiency of financial transactions and asset management.

Implementation:

- **Blockchain Network:** Hyperledger Iroha was chosen for its simplicity and ease of integration.
- **Digital Assets:** The platform supports the creation, management, and transfer of digital assets, including cryptocurrencies and other financial instruments.

Results:

- **Streamlined Transactions:** The blockchain-based platform has simplified and accelerated financial transactions.
- **Improved Asset Management:** Enhanced management and tracking of digital assets, increasing overall operational efficiency.

5. Hyperledger Besu: Enterprise Ethereum

Industry: Finance

Use Case: Consensys' Financial Services Solution

Overview: Consensys, a blockchain technology company, has implemented Hyperledger Besu to provide a private Ethereum blockchain for enterprise financial services.

Implementation:

- **Blockchain Network:** Hyperledger Besu, an Ethereum client designed for enterprise use, was employed to create a private blockchain network.
- **Smart Contracts:** The network supports the use of smart contracts to automate and manage complex financial transactions and agreements.

Results:

- **Increased Efficiency:** Automation of financial transactions through smart contracts reduces processing times and costs.

- **Enhanced Security:** The private blockchain offers robust security and compliance features tailored to financial services.

6. Hyperledger Burrow: Smart Contract Execution

Industry: Legal and Compliance

Use Case: ChainSafe's Legal Document Management

Overview: ChainSafe, a blockchain development company, used Hyperledger Burrow to create a platform for managing and executing legal agreements through smart contracts.

Implementation:

- **Blockchain Network:** Hyperledger Burrow was chosen for its integration with the Ethereum Virtual Machine (EVM) and support for smart contract execution.
- **Legal Contracts:** The platform allows for the creation, management, and automatic execution of legal contracts based on predefined conditions.

Results:

- **Efficient Legal Processes:** Streamlined the management of legal documents and automated contract execution.
- **Reduced Disputes:** The use of smart contracts ensures that agreements are executed as specified, reducing potential disputes.

7. Hyperledger Cactus: Cross-Blockchain Transactions

Industry: Interoperability

Use Case: Cross-Blockchain Transactions Platform

Overview: Hyperledger Cactus, a project aimed at solving blockchain interoperability challenges, is used to develop solutions for cross-blockchain transactions and interactions.

Implementation:

- **Blockchain Network:** Hyperledger Cactus provides a framework for integrating multiple blockchains, enabling transactions and data exchanges across different networks.

- **Integration:** The solution facilitates seamless interaction between various blockchain networks and enterprise systems.

Results:

- **Enhanced Interoperability:** Enables businesses to connect and interact with multiple blockchain networks, improving operational flexibility.
- **Unified Transactions:** Streamlines transactions and data flows across different blockchain systems, enhancing overall efficiency.

Conclusion

Hyperledger provides a range of blockchain frameworks suited for various industries and use cases. From supply chain management to digital identity and financial services, Hyperledger projects offer solutions that address specific business needs through secure, transparent, and efficient blockchain technology. Each case study demonstrates the versatility and impact of Hyperledger frameworks in real-world applications, showcasing their potential to transform industries and streamline operations.

8 Case Study on Other Blockchain platforms.

ertainly! Here's a selection of case studies showcasing different blockchain platforms beyond Hyperledger, highlighting their applications in various industries:

1. Ethereum: Decentralized Finance (DeFi) with Uniswap

Industry: Finance

Use Case: Uniswap

Overview: Uniswap is a decentralized exchange (DEX) protocol built on Ethereum, allowing users to trade Ethereum-based tokens directly from their wallets without needing a centralized intermediary.

Implementation:

- **Platform:** Ethereum
- **Smart Contracts:** Uniswap uses Ethereum's smart contract functionality to automate and manage trading, liquidity provision, and token swaps.
- **Automated Market Maker (AMM):** The protocol employs AMM algorithms to facilitate trading and liquidity provision.

Results:

- **Decentralization:** Uniswap has enabled trustless trading and liquidity provision, eliminating the need for centralized exchanges.
- **High Liquidity:** The platform has attracted substantial liquidity, making it one of the largest decentralized exchanges in terms of trading volume.

2. Bitcoin: Digital Gold with Bitcoin

Industry: Financial Investment

Use Case: Bitcoin as a Store of Value

Overview: Bitcoin is often referred to as "digital gold" due to its function as a store of value and hedge against inflation. It operates on a proof-of-work blockchain and has become a significant asset class for institutional and retail investors.

Implementation:

- **Platform:** Bitcoin
- **Security:** Utilizes a decentralized network and proof-of-work consensus mechanism to secure the blockchain.
- **Adoption:** Widely accepted by both individual and institutional investors as a digital asset and investment vehicle.

Results:

- **Market Value:** Bitcoin has achieved significant market capitalization and is considered a leading cryptocurrency.
- **Mainstream Adoption:** It is increasingly accepted by financial institutions and payment processors as a legitimate form of digital asset.

3. Ripple: Cross-Border Payments with RippleNet

Industry: Banking and Finance

Use Case: RippleNet

Overview: RippleNet is a blockchain-based payment platform developed by Ripple, designed to facilitate fast and low-cost cross-border transactions between banks and financial institutions.

Implementation:

- **Platform:** Ripple's custom blockchain protocol and XRP Ledger.
- **Technology:** Utilizes Ripple's XRP cryptocurrency for liquidity and transaction settlement.
- **Network:** Connects various financial institutions and payment providers for seamless transactions.

Results:

- **Faster Transactions:** RippleNet has significantly reduced transaction times from days to seconds.
- **Cost Efficiency:** Offers lower transaction fees compared to traditional banking systems.

4. EOS: Scalable DApps with EOSIO

Industry: Decentralized Applications (DApps)

Use Case: Voice

Overview: Voice is a decentralized social media platform built on EOSIO, aimed at providing a transparent and fair platform for content creators and users.

Implementation:

- **Platform:** EOSIO
- **Scalability:** EOSIO's high throughput and scalability support the operation of the Voice platform with fast transaction speeds.
- **Tokenization:** Uses EOS tokens for incentivizing and rewarding user engagement and content creation.

Results:

- **User Engagement:** Voice aims to enhance user engagement by ensuring fair content distribution and compensation.
- **Scalability:** EOSIO's architecture supports high transaction volumes, ensuring a smooth user experience.

5. Tezos: Governance and Upgradability with Tezos

Industry: Blockchain Governance

Use Case: Nomadic Labs' Tezos

Overview: Nomadic Labs, a research and development entity, has used Tezos to develop smart contracts and decentralized applications with a focus on on-chain governance and upgradability.

Implementation:

- **Platform:** Tezos
- **Governance:** Tezos employs an on-chain governance mechanism that allows stakeholders to propose and vote on protocol upgrades.
- **Smart Contracts:** Supports formal verification of smart contracts to ensure correctness.

Results:

- **Continuous Improvement:** The on-chain governance model facilitates ongoing improvements and updates to the Tezos protocol without hard forks.
- **Security:** Formal verification of smart contracts enhances the security and reliability of deployed applications.

6. Chainlink: Decentralized Oracles with Chainlink

Industry: Smart Contracts and Data

Use Case: Chainlink Network

Overview: Chainlink is a decentralized oracle network that provides smart contracts with access to real-world data, enabling them to interact with external systems and APIs.

Implementation:

- **Platform:** Ethereum and other blockchain platforms.
- **Oracles:** Chainlink uses a network of decentralized oracles to fetch and deliver external data to smart contracts.
- **Data Integrity:** Ensures the accuracy and reliability of the data provided to smart contracts.

Results:

- **Expanded Functionality:** Chainlink enables smart contracts to interact with a wide range of external data sources, expanding their use cases.
- **Adoption:** Widely adopted by various decentralized applications and blockchain platforms for reliable data feeds.

7. Corda: Trade Finance with Corda

Industry: Trade Finance

Use Case: Voltron Trade Finance

Overview: Voltron, a trade finance platform, uses Corda to streamline and digitize the trade finance process, enabling faster and more efficient trade transactions between parties.

Implementation:

- **Platform:** Corda
- **Private Transactions:** Corda's design focuses on privacy and confidentiality, allowing only authorized parties to access transaction details.
- **Smart Contracts:** Uses Corda's smart contracts to automate trade finance workflows and reduce paperwork.

Results:

- **Efficiency:** The platform has improved the efficiency of trade finance processes by reducing transaction times and paperwork.
- **Cost Savings:** Streamlined processes have led to cost savings for businesses involved in trade finance.

8. Algorand: Digital Currency with Algorand

Industry: Digital Currency

Use Case: Algorand's Digital Asset Issuance

Overview: Algorand is a blockchain platform designed for high-speed and low-cost transactions. It supports the issuance of digital assets and the creation of decentralized applications.

Implementation:

- **Platform:** Algorand
- **Consensus Mechanism:** Utilizes a pure proof-of-stake (PPoS) consensus protocol to achieve high performance and scalability.
- **Digital Assets:** Supports the creation and management of custom digital assets and tokens.

Results:

- **Scalability:** Algorand's protocol supports a high transaction throughput, making it suitable for digital asset issuance and other applications.
- **Low Costs:** Transaction fees on the Algorand network are minimal, enhancing its appeal for various digital asset use cases.

Conclusion

These case studies highlight the diverse applications of different blockchain platforms across various industries. Each platform offers unique features and capabilities tailored to specific use cases, ranging from financial transactions and decentralized applications to digital identity and trade finance. These examples demonstrate how blockchain technology can drive innovation and efficiency in different sectors.

9. Creating a blockchain Application as a mini project

Table Of Contents

1.Abstract
2.Introduction
3. Implementation
4.Result
5.Conclusion

Implementation

Creating a blockchain application as a mini-project is a great way to understand the practical aspects of blockchain technology. For this mini-project, we'll build a simple Decentralized Voting Application using Ethereum and Solidity. This example will cover:

1. Setting Up the Development Environment
2. Developing the Smart Contract
3. Deploying the Smart Contract
4. Building a Frontend Application
5. Interacting with the Smart Contract

Sample Mini Project

1. Setting Up the Development Environment

1.1. Install Required Tools:

- **Node.js:** Download and install from Node.js official site.

  Truffle Suite: A development framework for Ethereum.
  bash
  Copy code
  npm install -g truffle

- **Ganache:** A personal blockchain for Ethereum development.
  - **Download Ganache UI:** From Ganache official site.

    Install Ganache CLI: If you prefer command-line interface.
    bash
    Copy code
    npm install -g ganache-cli

  -

1.2. Create a New Truffle Project:

bash
Copy code
mkdir voting-app
cd voting-app
truffle init

2. Developing the Smart Contract

2.1. Create a Smart Contract:

In the contracts directory, create a new file Voting.sol:

solidity
Copy code

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract Voting {
    // Declare state variables
    address public owner;
    mapping(address => bool) public voters;
    mapping(string => uint) public votesReceived;
    string[] public candidateList;

    // Constructor
    constructor(string[] memory candidateNames) {
        owner = msg.sender;
        candidateList = candidateNames;
    }

    // Modifier to allow only the owner to add voters
    modifier onlyOwner() {
        require(msg.sender == owner, "Only the owner can perform this action");
        _;
    }

    // Function to add a voter
    function addVoter(address voter) public onlyOwner {
        voters[voter] = true;
```

```
    }

    // Function to vote for a candidate
    function vote(string memory candidate) public {
        require(voters[msg.sender], "You must be a registered
voter to vote");
        require(isValidCandidate(candidate), "Invalid candidate
name");
        votesReceived[candidate]++;
    }

    // Helper function to check if the candidate is valid
        function isValidCandidate(string memory candidate)
public view returns(bool) {
        for (uint i = 0; i < candidateList.length; i++) {
            if (keccak256(abi.encodePacked(candidateList[i]))
== keccak256(abi.encodePacked(candidate))) {
                return true;
            }
        }
        return false;
    }

    // Function to get the vote count for a candidate
      function getVoteCount(string memory candidate) public
view returns(uint) {
        return votesReceived[candidate];
    }
}
```

2.2. Compile the Smart Contract:

bash
Copy code

```
truffle compile
```

3. Deploying the Smart Contract

3.1. Create a Migration Script:

In the migrations directory, create a new file 2_deploy_contracts.js:

javascript
Copy code

```
const Voting = artifacts.require("Voting");

module.exports = function(deployer) {
  const candidateNames = ["Alice", "Bob", "Charlie"];
  deployer.deploy(Voting, candidateNames);
};
```

3.2. Start Ganache:

- **Ganache UI:** Open the Ganache application to start a local blockchain.

Ganache CLI:
bash
Copy code

```
ganache-cli
```

-

3.3. Deploy the Smart Contract:

bash
Copy code

```
truffle migrate --network development
```

4. Building a Frontend Application

4.1. Set Up the Frontend:

Create a new directory client inside your project and initialize it with a basic HTML and JavaScript setup:

bash

Copy code

```
mkdir client
cd client
npm init -y
npm install web3
```

4.2. Create index.html:

html

Copy code

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
        <meta  name="viewport"  content="width=device-width,
initial-scale=1.0">
    <title>Voting App</title>
</head>
<body>
    <h1>Voting Application</h1>
    <div>
        <h2>Vote for a Candidate</h2>
                        <input   type="text"   id="candidateName"
placeholder="Candidate Name">
        <button onclick="vote()">Vote</button>
    </div>
    <div>
        <h2>Get Vote Count</h2>
                    <input   type="text"   id="candidateNameCount"
placeholder="Candidate Name">
                     <button   onclick="getVoteCount()">Get   Vote
Count</button>
        <p id="voteCountResult"></p>
    </div>
                                                                <script
src="https://cdn.jsdelivr.net/npm/web3@latest/dist/web3.min
.js"></script>
```

```
    <script src="app.js"></script>
</body>
</html>
```

4.3. Create app.js:

```
javascript
Copy code
const Web3 = require('web3');
const web3 = new Web3('http://localhost:8545'); // Connect to Ganache

const contractAddress = 'YOUR_CONTRACT_ADDRESS';
const contractABI = [ /* ABI generated by Truffle */ ];

const contract = new web3.eth.Contract(contractABI, contractAddress);
const accounts = await web3.eth.getAccounts();

async function vote() {
                    const candidateName = document.getElementById('candidateName').value;
          await contract.methods.vote(candidateName).send({ from: accounts[0] });
    alert('Vote cast successfully');
}

async function getVoteCount() {
                    const candidateName = document.getElementById('candidateNameCount').value;
                    const count = await contract.methods.getVoteCount(candidateName).call();
     document.getElementById('voteCountResult').innerText = `Vote Count: ${count}`;
}
```

4.4. Serve the Frontend:

You can use a simple HTTP server to serve your frontend files:

bash
Copy code
```
npm install -g http-server
http-server client
```

5. Interacting with the Smart Contract

1. **Open Ganache** and **start your local blockchain**.
2. **Deploy your smart contract** using Truffle.
3. **Run your frontend application** using http-server.
4. **Open index.html** in your browser and interact with the voting application by adding votes and checking vote counts.

Summary

- **Developed a smart contract** for a voting application using Solidity.
- **Deployed the smart contract** to a local Ethereum blockchain using

Ganache and Truffle.

- **Built a simple frontend** application to interact with the smart contract using Web3.js.

This project provides a practical introduction to developing and deploying blockchain applications. You can extend this project by adding more features, such as user authentication or advanced voting analytics.